



第2章 指令系统的设计

- 2.1 [指令系统结构的分类](#)
- 2.2 [寻址方式](#)
- 2.3 [指令系统的设计与优化](#)
- 2.4 [指令系统的发展和改进](#)
- 2.5 [操作数的类型和大小](#)
- 2.6 [MIPS指令系统结构](#)



➤ 回顾：程序执行的CPU时间可以写成

$$\text{CPU时间} = IC \times CPI \times \text{时钟周期时间}$$

1. 指令系统(IS)的目的

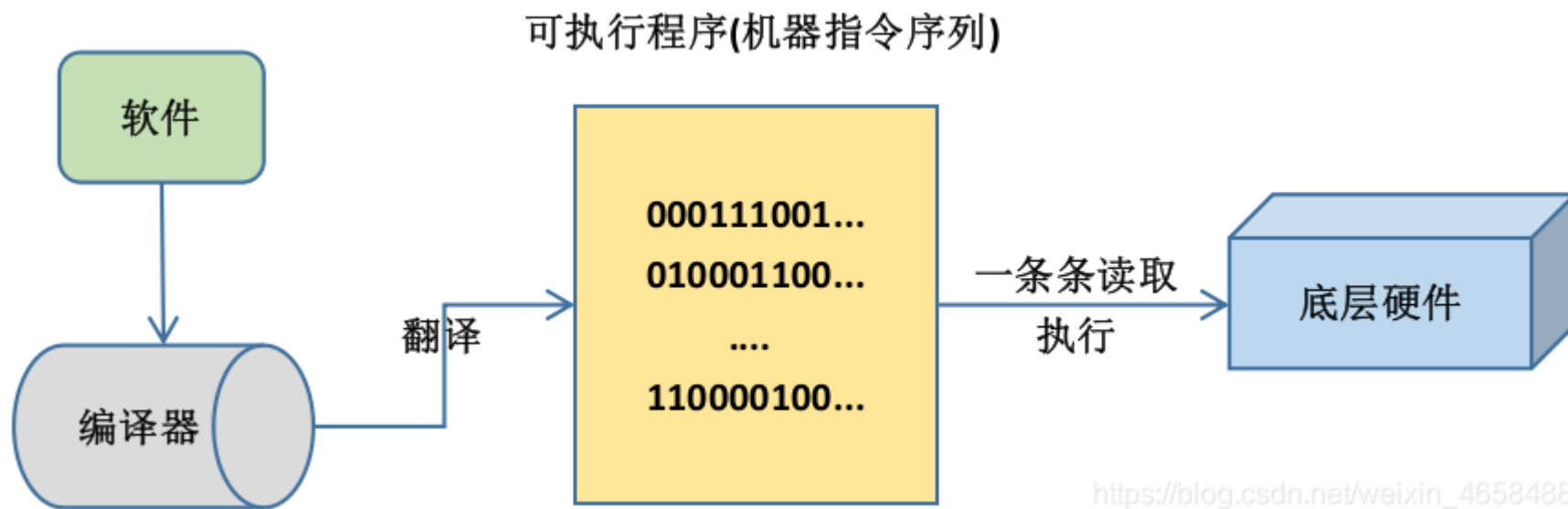
➤ 运行程序，较快地运行程序

- ✓ **可改进的地方（根据上述公式）**：IC（指令条数），CPI（每条指令平均执行的时钟周期数，取决于计算机组成和指令系统的结构（例如要不要流水线技术、要不要cache技术？要与不要对CPI的影响）；）
- ✓ **时钟周期时间**：与硬件频率相关，目前已接近其物理极限，暂不考虑



2.1 指令系统结构的分类

- 计算机指令系统（Instruction System, IS）：一台计算机能够直接识别与执行的机器指令的集合。



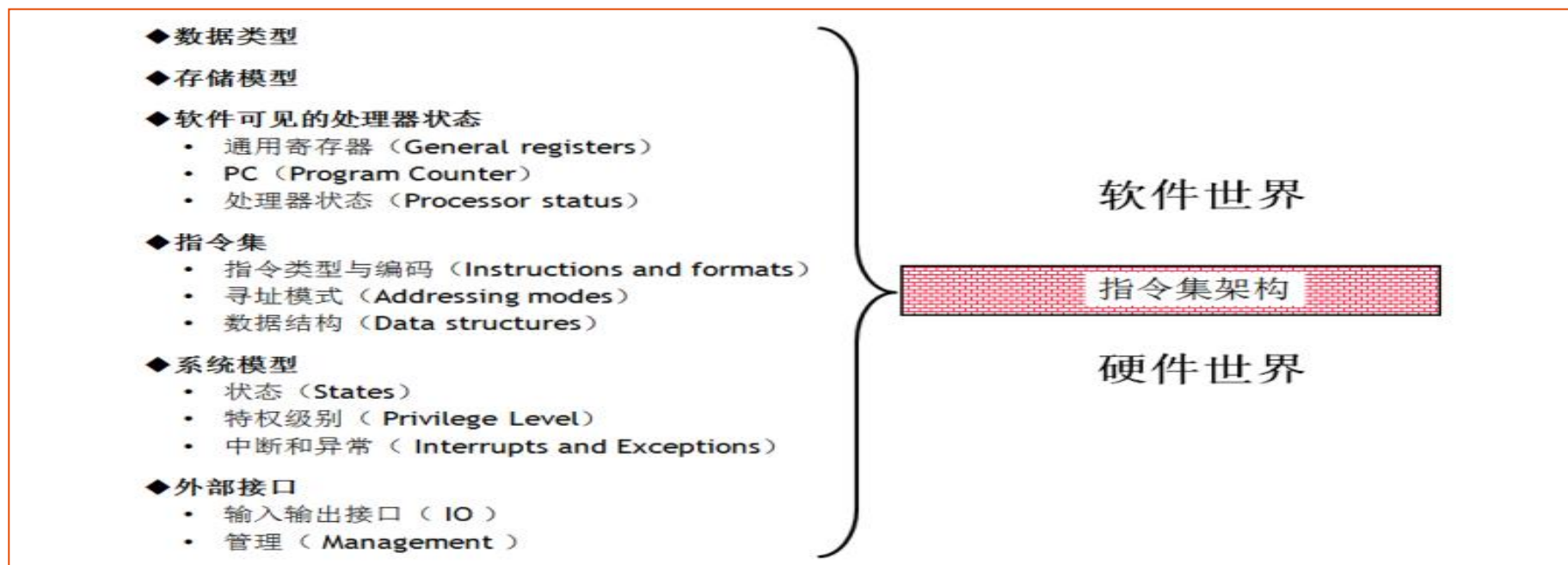
计算机系统工作流程



2.1 指令系统结构的分类

➤ 指令系统结构（Instruction Set Architecture, ISA）概念

- ◆ 又名指令集架构，或计算机架构，是计算机的抽象模型
- ◆ ISA定义了支持的数据类型、寄存器、管理主存的硬件支持、基本功能（寻址模式、虚存等）、指令格式，以及ISA一系列实现的I/O模型（交互模式等）

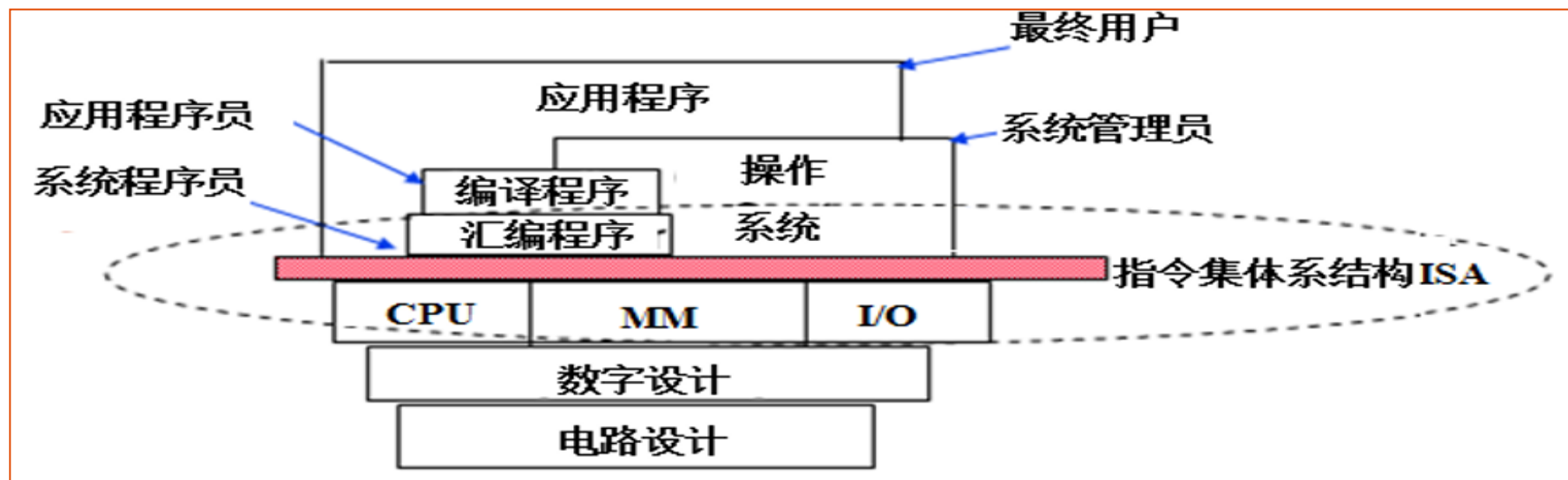




2.1 指令系统结构的分类

➤ 指令系统(ISA): 软硬件的分界面

- ◆ 定义了软硬件交互的“协约或接口”
- ◆ 提供了一种机制：软件告诉硬件该执行什么操作的机制
- ◆ 是软硬件交流的工具（使用同一种指令系统）
- ◆ 承担软硬件接口的重任，ISA设计与扩展必须遵循严格的工程化





2.1 指令系统结构的分类

➤ 流行指令集

- ◆ Intel 公司的 x86 指令集、ARM 公司的 ARM 指令集，开源的 RISC-V 指令集、MIPS指令集等
 - ✓ X86架构是微处理器执行的计算机语言指令集，指一个intel通用计算机系列的标准编号缩写，也标识一套通用的计算机指令集合。
 - ✓ ARM架构，进阶精简指令集机器（Advanced RISC Machine），一个32位精简指令集（RISC）处理器架构。ARM处理器广泛使用在嵌入式系统设计。
 - ✓ RISC-V是一种指令集，RISC系列指令集的第五代产品。被CPU读取到内存后，指导计算机运行的指令集合。
 - ✓ MIPS指令集，经典 RISC-ISA，教科书最经典 RISC 指令集。



2.1 指令系统结构的分类

➤ CPU与ISA的关系

- ◆ 先有ISA，然后才有CPU的设计
- ◆ CPU是ISA的具体实现

➤ 流行指令集架构ISA

◆ CISC: x86

✓ Intel 公司的 x86 指令集

- ◆ 有两家厂商基于**X86**生产对应的**CPU**: **Intel**和**AMD**（授权公司）。

◆ RISC: ARM、MIPS、RISC-V、LongArch龙芯系列

✓ 都有自身对应的ISA



2.1 指令系统结构的分类

指令系统结构可以按指令长短分，也可以按 RISC/CISC 分

教材- 指令系统结构的分类，依据是操作数存放位置。

- 1、区别不同指令系统结构的**主要因素**
CPU中用来存储操作数的存储单元的类型
- 2、CPU中用来存储操作数的存储单元（指令使用的数据）
 - ◆ 堆栈
 - ◆ 累加器
 - ◆ 通用寄存器组



2.1 指令系统结构的分类

➤ 3、根据上述内容将指令系统的结构分为三种类型

- ◆ 堆栈结构：操作数隐式存于栈顶
- ◆ 累加器结构：其中一个操作数固定在累加器
- ◆ 通用寄存器结构

根据操作数的来源不同，又可进一步分为：

- ✓ 寄存器-存储器结构（RM结构）：（操作数可以来自存储器）
- ✓ 寄存器-寄存器结构（RR结构）：（所有操作数都是来自通用寄存器组）

也称为load-store结构，这个名称强调：只有load指令和store指令能够访问存储器（加快访问速度）

RM和RR的区别：做运算的时候，能不能直接取内存的数据

例如RM结构：执行ADD R1,X
直接从内存单元X中取数据参与运算

例如RR结构：要执行ADD R1,X
首先 IW R2, X
然后 ADD,R1,R2



2.1 指令系统结构的分类

- 4、对于不同类型的结构，操作数的位置、个数以及操作数的给出方式（显式或隐式）也会不同。
- ◆ 显式给出：用指令字中的操作数字段给出
 - ◆ 隐式给出：使用事先约定好的单元（例如堆栈结构中，栈顶操作数都是隐式的；累加器其中一个操作数是隐式的）



2.1 指令系统结构的分类

例：表达式 $Z=X+Y$ 在4种类型指令系统结构上的代码。

假设：X、Y、Z均保存在存储器单元中，并且不能破坏X和Y的值。

堆 栈	累加器	寄存器（RM型）	寄存器（RR型）
push X	存X → 累加器ACC load X	load R1, X	load R1, X
push Y	add Y	add R1, Y	load R2, Y
add	ACC ← ACC + Mem[Y] store Z	运算指令直接访问内存Y! store R1, Z	add R3, R1, R2
栈顶两个元素 弹出相加，结 果压回栈顶 pop Z			store R3, Z

add Y 结果保存在累加器中

分析：指令条数？执行运算指令时操作数是否访存？



2.1 指令系统结构的分类

5. 通用寄存器型结构

- ◆ 现代指令系统结构的主流
- ◆ 在灵活性和提高性能方面有明显的优势
 - ✓ 寄存器的访问 速度比存储器快。
 - ✓ 对编译器而言，能更加容易、有效地分配和使用寄存器。
 - ✓ 寄存器可以用来存放变量。

（1）减少对存储器的访问，加快程序的执行速度；（因为寄存器比存储器快）（例如for循环中的自变量）

（2）用更少的地址位（相对于存储器地址来说）来对寄存器进行寻址，从而有效地减少程序的目标代码的大小。

在 RISC 中，存储指令一般通过寄存器间接寻址的方式访问存储器

- ✓ 堆栈型和累加器型优点指令字短，占用空间小
- ✓ 但堆栈难以灵活访问，栈顶访问受限，累加器结构，一个操作数在内存需要频繁访问存储器



2.1 指令系统结构的分类

6. 根据ALU指令（算术和逻辑指令）的操作数的两个特征对通用寄存器型结构进一步细分

◆ ALU指令的操作数个数

✓ 3个操作数的指令

两个源操作数、一个目的操作数

✓ 2个操作数的指令

其中一个操作数既作为源操作数，又作为目的操作数。

◆ ALU指令中存储器操作数的个数

可以是0~3中的某一个，为0表示没有存储器操作数。

7. ALU指令中操作数个数和存储器操作数个数的典型组合

ALU指令中存储器操作数的个数	ALU指令中操作数的最多个数	结构类型	机器实例
0	3	RR	MIPS, SPARC, Alpha, PowerPC, ARM
1	2	RM	IBM 360/370, Intel 80x86, Motorola 68000
	3	RM	IBM 360/370
2	2	MM	VAX
3	3	MM	VAX

表中(m, n)表示指令的n个操作数中有m个存储器操作数。

指令系统结构类型	优 点	缺 点
寄存器—寄存器型 (0, 3)	指令字长固定，指令结构简洁，是一种简单的代码生成模型，各种指令的执行时钟周期数相近。	与指令中含存储器操作数的指令系统结构相比，指令条数多，目标代码不够紧凑，因而程序占用的空间比较大。
寄存器—存储器型 (1, 2)	可以在ALU指令中直接对存储器操作数进行引用，而不必先用load指令进行加载。容易对指令进行编码，目标代码比较紧凑。	指令中的两个操作数不对称。在一条指令中同时对寄存器操作数和存储器操作数进行编码，有可能限制指令所能够表示的寄存器个数。指令的执行时钟周期数因操作数的来源（寄存器或存储器）不同而差别比较大。
存储器—存储器型 (2, 2) 或 (3, 3)	目标代码最紧凑，不需要设置寄存器来保存变量。	指令字长变化很大，特别是3操作数指令。而且每条指令完成的工作也差别很大。对存储器的频繁访问会使存储器成为瓶颈。 这种类型的指令系统结构现在已不用了。

存储器访问：存储指令的相关性隐藏得很深，机器没办法预先掌握存储指令之间的相关性（例如读后写、写后读、写后写等冲突）



2.2 指令系统的寻址方式

➤ **寻址方式**：指令系统中如何形成所要访问的数据的地址。

- ◆ 寻找指令

- ✓ 顺序与跳转指令

- ◆ 寻找操作数

- ✓ 寻址方式可以指明指令中的操作数是一个常数、一个寄存器操作数或者是一个存储器操作数。
- ✓ 对于存储器操作数来说，由寻址方式确定的存储器地址称为**有效地址**。

1、一些操作数寻址方式的符号表达

- ◆ $\leftarrow$ ：赋值操作

- ◆ Mem：存储器

- ◆ Regs：寄存器组

- ◆ 方括号：表示内容

- ✓ Mem[]：存储器的内容

- ✓ Regs[]：寄存器的内容

- ✓ Mem[Regs[R1]]：以寄存器R1中的内容作为地址的存储器单元中的内容

寻址方式	指令实例	含 义
寄存器寻址	Add R4 , R3	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Regs}[\text{R3}]$
立即值寻址	Add R4 , #3	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + 3$
偏移寻址	Add R4 , 100(R1)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[100 + \text{Regs}[\text{R1}]]$
寄存器间接寻址	Add R4 , (R1)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[\text{Regs}[\text{R1}]]$
索引寻址	Add R3 , (R1 + R2)	$\text{Regs}[\text{R3}] \leftarrow \text{Regs}[\text{R3}] + \text{Mem}[\text{Regs}[\text{R1}] + \text{Regs}[\text{R2}]]$
直接寻址或绝对寻址	Add R1 , (1001)	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[1001]$
存储器间接寻址	Add R1 , @(R3)	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[\text{Mem}[\text{Regs}[\text{R3}]]]$
连续存放	自增寻址	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[\text{Regs}[\text{R2}]]$ $\text{Regs}[\text{R2}] \leftarrow \text{Regs}[\text{R2}] + d$
	自减寻址	$\text{Regs}[\text{R2}] \leftarrow \text{Regs}[\text{R2}] - d$ $\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[\text{Regs}[\text{R2}]]$
	缩放寻址	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[100 + \text{Regs}[\text{R2}] + \text{Regs}[\text{R3}] * d]$ 力画@



2.2 指令系统的寻址方式

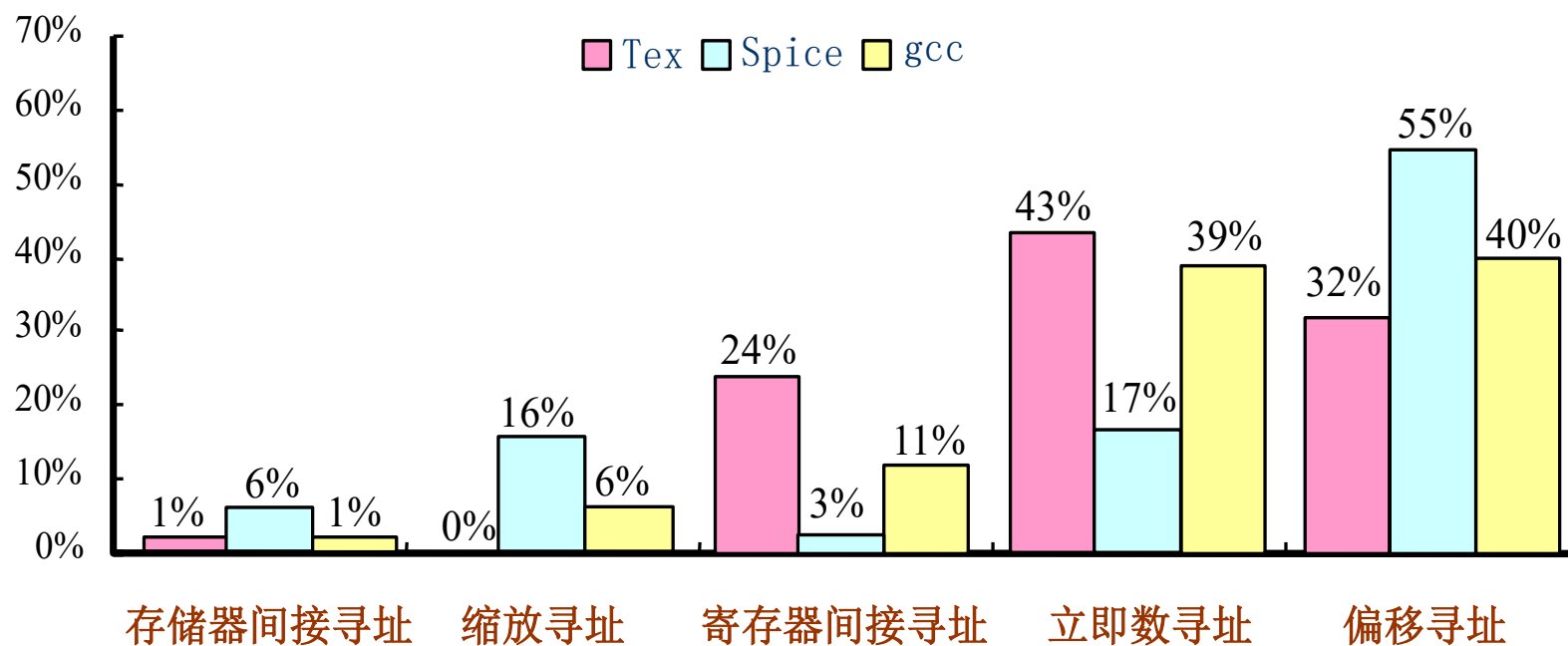
- 2、操作数寻址为什么采用多种寻址方式（由上页图分析）
 - ◆ 可以显著地减少程序的指令条数
 - ◆ 缩短指令长度、扩大寻址空间，提高编程灵活性
 - ◆ 但可能增加计算机的实现复杂度以及指令的CPI
- 需要有选择的使用寻址方式，如下图与表，分析寻址方式的使用情况，进而如何对操作数寻址方式进行优化设计



2.2 指令系统的寻址方式

3. 各种操作数寻址方式的使用情况统计结果

在VAX机器（经典的CISC机器）上运行gcc、Spice和Tex 基准程序



结论：立即数寻址方式和偏移寻址方式的使用频度最高。

联系：以经常性事件为重点-----定量原理



2.2 指令系统的寻址方式

4. 立即数寻址方式

➤ 立即数寻址方式的使用频度

指令类型	使用频度	
	整型平均	浮点平均
load指令	23%	22%
ALU指令	25%	19%
所有指令	21%	16%

因此：RISC 硬件优先高效实现偏移寻址、立即数寻址；对于使用频率很低的存储器间接寻址，不在硬件实现，改为由编译器通过多条简单指令软件完成，简化硬件，保证指令规整，利于流水线。

大约1/4的load指令和ALU指令采用了立即数寻址。

联系：以经常性事件为重点-----定量原理



2.2 指令系统的寻址方式

5. 寻址方式中涉及到的物理地址空间需要注意的问题：物理地址空间的信息如何存放？

如何在存储器中存放不同宽度的信息？

以IBM370为例子进行讨论。

- ✓ 信息有字节、半字（双字节）、单字（4字节）和双字（8字节）等宽度。
- ✓ 主存宽度为8个字节。采用按字节编址，各类信息都是用该信息的首字节地址来寻址。
- ◆ 允许它们任意存储
 - ✓ 很可能会出现一个信息跨存储字边界而存储于两个存储单元中



2.2 指令系统的寻址方式

◆ 存储字长与数据字长的概念

- ✓ 存储字长：主存的一个存储单元所存储的二进制位数。（和数据总线对应）
- ✓ 数据字长（简称字长）：计算机一次能处理的二进制数的位数。存储字长与数据字长不一定相同，如字长32位，则半字长16位，双字长64位。

◆ 地址访问模式

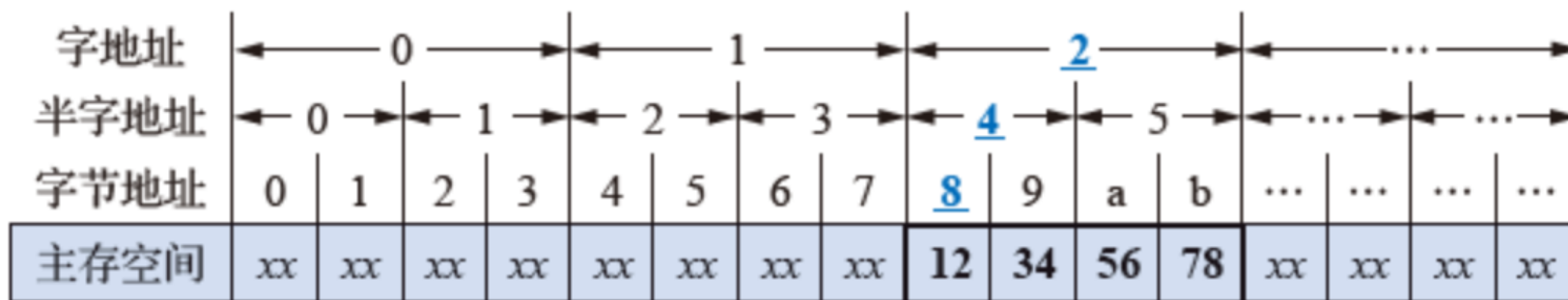
- ✓ 存储字长都是字节的整倍数，主存通常按字节进行编址。
- ✓ 以32位计算机为例，主存既可以按字节访问，也可以按16位半字访问，还可以按照32位的字进行访问。
- ✓ 按照访问存储单元的大小，主存地址可以分为字节地址、半字地址、字地址。



2.2 指令系统的寻址方式

◆地址访问模式

- ✓ 8号字节地址对应的半字地址为4，字地址为2，注意这3个地址在逻辑空间的起始位置都是8号字节单元，区别只是存储单元的大小不同
 - ◆ 例如把4个字节当成一个整体访问，编号是字地址2，此时CPU发出2号地址，硬件自动换算真实字节地址=2×4=8
- ✓ 底层硬件地址永远都是字节地址，字地址等只是访问时的逻辑编号而已



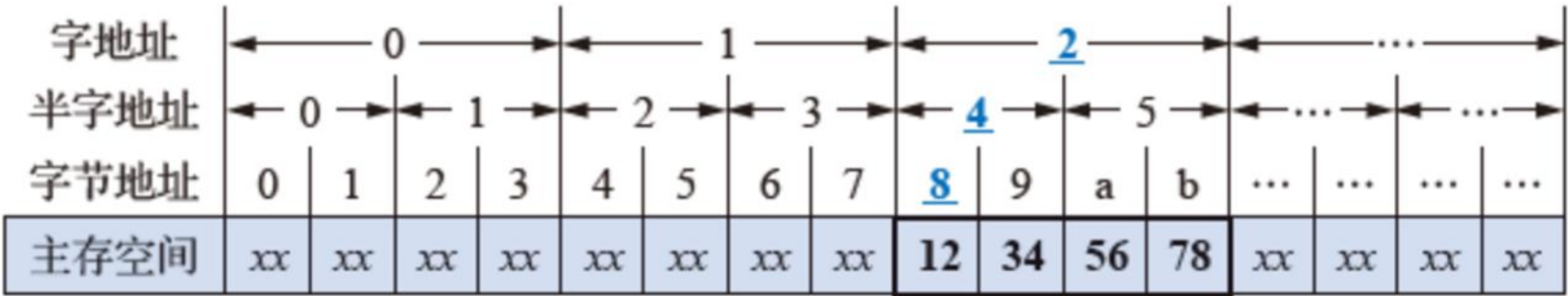
不同主存地址访问模式



2.2 指令系统的寻址方式

◆大端和小端方式（小端模式—低低高高）

- ✓ 访问2 号**字存储单元**时，如果按小端方式访问得到的数据是0x78563412，而如果按照大端方式访问得到的数据则是0x12345678，采用大、小端方式对数据进行存放的主要区别在于字节的存放顺序。
- ✓ 采用大端方式进行数据存放符合人类的正常思维，而采用小端方式进行数据存放有利于计算机处理。



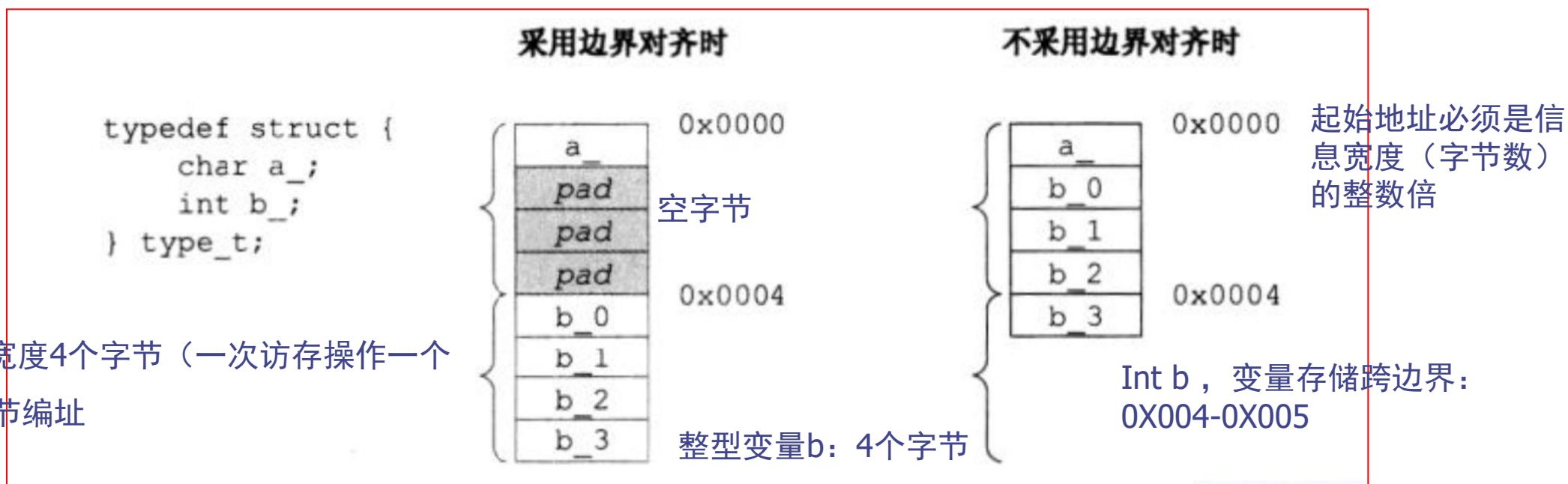
不同主存地址访问模式



2.2 指令系统的寻址方式

- ◆ 信息宽度不超过主存宽度，信息必须存放在一个存储字内，不能跨边界。
 - ✓ **必须做到：**信息在主存中存放的起始地址必须是该信息宽度（字节数）的整数倍—— **信息存储的整数边界概念**
 - ✓ **目的：**处理器为了提高处理性能而对存取数据的起始地址所提出的一种要求，编译器最大限度地满足处理器对边界对齐的要求。

前提条件：主存宽度4个字节（一次访存操作一个4B字），且按字节编址





2.2 寻址方式

◆ 字长32位，则半字长16位，双字长64位。则按边界对齐方式的存储要求满足以下条件

✓ 半字信息的起始地址为： $\times \dots \times \times \times 0$ （2字节的整数倍）



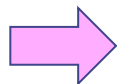
✓ **半字：两个字节**，存储单元都是以字节为单位，那么要索引一个数据，需要连续的2个字节才行，比如，0x0000_0000 ~ 0x0000_0001这2个单元存储一个数据，紧接着0x0000_0002 ~ 0x0000_0003这2个单元存储第二个数据，依此类推，每个数据的存储**起始地址**为：

0x0000_0000

0x0000_0002

0x0000_0004

.....



对应二进制地址：

00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000010
00000000 00000000 00000000 00000100

结论：起始地址最后一个二进制位都为“0”，该地址才是2的幂次方，即2的整数倍



2.2 寻址方式

- ◆ 信息宽度不超过主存宽度信息必须存放在一个存储字内，不能跨边界。
 - ✓ 字长32位，则半字长16位，双字长64位。则按边界对齐方式的存储要求满足以下条件（地址：二进制）
 - ◆ 字节信息的起始地址为： $\times \dots \times \times \times \times$ （每个字节地址都用上了）
 - ◆ 半字信息的起始地址为： $\times \dots \times \times \times 0$ （2字节的整数倍）
 - ◆ 单字信息的起始地址为： $\times \dots \times \times 0 0$ （4字节的整数倍）
 - ◆ 双字信息的起始地址为： $\times \dots \times 0 0 0$ （8字节的整数倍）
 - ✓ **代价：存在存储空间的浪费，但保证访问速度。**
 - ◆ 按照指定空间存储，确保一个字长数据的读/写只需要一次存储器访问，效率提高，但有时存储空间浪费。
 - ✓ **结果：存储空间换取速度**



2.2 寻址方式

主存数据宽度64位，若字长32位，则半字长16位，双字长64位

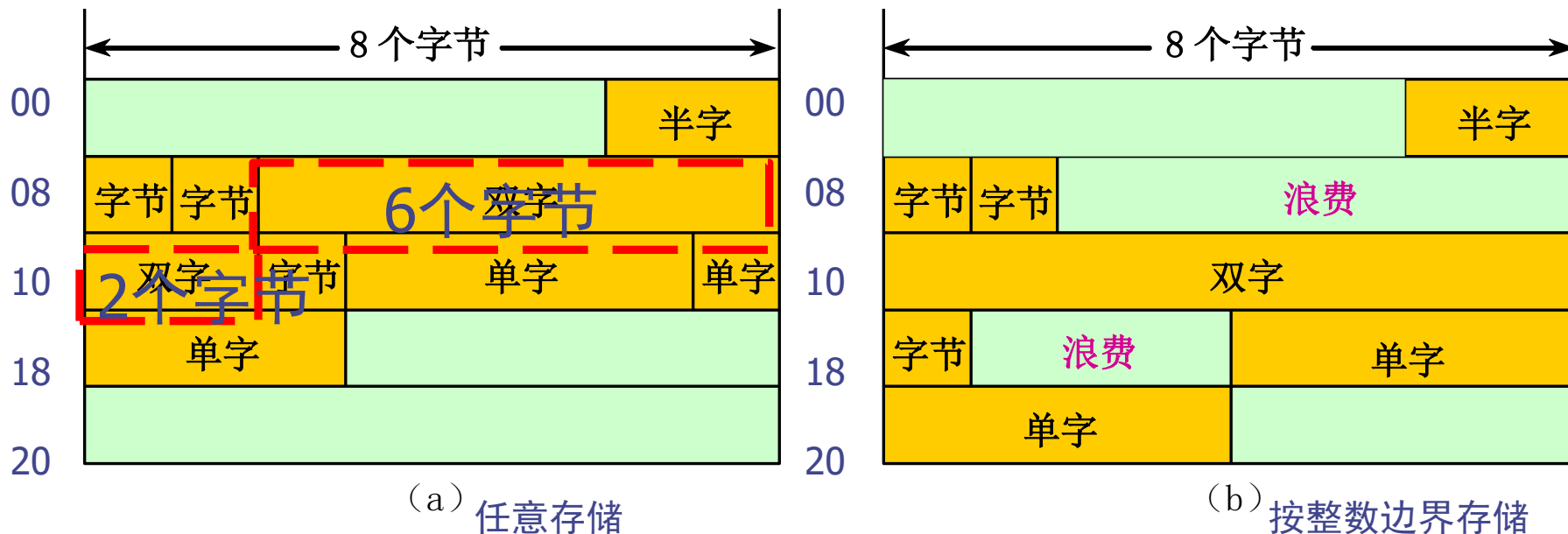


图 2.2 各种宽度的信息的存储

这种方式对存储空间的利用率最高，
但对跨单元的存储访问周期增加

有效提升了存储速度。但注意这种方式
会造成空间的浪费，需要折中考虑

需要折中考虑

1、不需要编址的数据存储空间是（ ）。

- ☐ A CPU中的通用寄存器
- ☐ B 主存储器
- ☐ C I/O接口中的寄存器
- ☒ D 堆栈

提交

2、信息按整数边界存储的主要优点是（ ）。

- ☒ A 访存速度快
- ☐ B 节约主存单元
- ☐ C 指令字的规整化
- ☐ D 指令的优化

提交

3、某机主存按字节编址，存储字长 32 位。系统规定：当信息宽度不超过主存宽度时，信息必须存放在一个存储字内，不能跨字边界。下列说法正确的是（）

2个字节，起始地址 3，占用地址 3、4，跨越了字边界（0~3 和 4~7）

- ☐ A 16 位操作数可以存放在字节地址 3 开始的位置
- ☒ B 16 位操作数可以存放在字节地址 4 开始的位置
- ☐ C 24 位操作数可以存放在字节地址 2 开始的位置
- ☐ D 32 位操作数可任意起始字节地址存放

提交



2.3 指令系统的设计与优化

2.3.1 指令系统设计的基本原则

➤ 指令系统的设计

- ◆ 首先考虑所应实现的基本功能，**划分原硬件的边界**：确定哪些基本功能应该由硬件实现，哪些功能由软件实现比较合适。

- ◆ **包括**

- ✓ 指令的功能设计：需要提供哪些指令？
- ✓ 指令格式的设计：指令编码在内存存放格式？操作码地址码字段分配？指令定长？

➤ 在确定哪些基本功能用硬件来实现时，主要考虑**3个因素**：速度、成本、灵活性（算法扩展等）。



2.3 指令系统的设计与优化

2.3.1 指令系统设计的基本原则

- ✓ 下表为许多指令系统结构都包含的一些指令类型
 - ◆ 前4类属于通用计算机系统的基本指令
 - ◆ 对于最后4种类型的操作（扩展指令），不同指令系统结构的支持大不相同。

前 4 种	算术和逻辑运算	浮点	后 4 种
	数据传输	十进制	
	控制	字符串	
	系统	图形	



2.3 指令系统的设计与优化

2.3.1 指令系统设计的基本原则

几乎全部硬件实现

可以硬件实现，也可以由编译器用软件子程序模拟完成

操作类型	实 例
算术和逻辑运算	算术运算和逻辑操作：加，减，乘，除，与，或等
数据传输	RISV:load, store; x86: mov
控制	分支，跳转，过程调用和返回，自陷等
系统	操作系统调用，虚拟存储器管理等
浮点	浮点操作：加，减，乘，除，比较等
十进制	十进制加，十进制乘，十进制到字符的转换等
字符串	字符串移动，字符串比较，字符串搜索等
图形	像素操作，压缩/解压操作等

可硬件可软件模拟

软件完成

软件完成



2.3 指令系统的设计与优化

➤ 设计原则：规整性、正交性、高效率、兼容性等

◆ **规整性**：在一个有限可用的存储空间内，对于任何可解的问题，编制计算程序时，指令系统所提供的指令足够使用。主要包括对称性和均匀性。

✓ **对称性**：所有与指令系统有关的存储单元的使用、操作码的设置等都是对称的。

例如：在操作码的设置上，如果设置了A—B（减法）的指令，就应该也设置B—A的指令。

✓ **均匀性**：同一操作尽量为各种组合设置对应的指令，相似的操作要有相同的规定。

例如：对字的操作同对字节的操作相同，操作后生成的条件码要一样；每个通用寄存器的用法要相同。



2.3 指令系统的设计与优化

2.3.1 指令系统设计的基本原则

➤ 设计原则：规整性、正交性、高效率、兼容性

- ◆ **正交性**：在指令中各个不同含义的字段，如操作类型、数据类型、寻址方式字段等，在编码时应互不相关、相互独立。（例如：浮点运算、定点运算，寄存器的选择不受限，任意访问）；例如**加法、减法、移位**，都可以随便搭配**寄存器寻址、立即数寻址、基址寻址**等。
- ◆ **高效率**：指指令的执行速度快、使用频度高。
- ◆ **兼容性**：主要是要实现向后兼容，指令系统可以增加新指令，但不能删除指令或更改指令的功能。
- ◆ **可扩充性**：有一定数量的冗余操作码，便于后续扩充新指令。

➤ 规整性、正交性：编译系统要求

➤ 高效率、兼容性：系统设计者要求



2.3 指令系统的设计与优化

4. 在设计指令系统时，有两种截然不同的设计策略。

（产生了两类不同的计算机系统：CISC和RISC）

◆ CISC（复杂指令系统计算机）--单条指令做复杂功能、微程序实现

- ✓ 增强指令功能，单条指令功能增加（让一条指令做更多事情）
- ✓ 容易编程（汇编语言），指令代码少
- ✓ 适用于商业（向下兼容）
- ✓ 设计周期长，准确性难以保障
- ✓ 需要大量硬件支持
- ✓ 大多复杂指令使用频率低，资源浪费
- ✓ CPI值较大，变长指令，不易于流水
- ✓ 2/8规律
- ✓ 灵活性强，若新增或修改复杂指令，电路不动，只修改微程序

◆ 实现方式：微程序控制技术（减少电路的重新设计、只需修改微程序）



2.3 指令系统的设计与优化

2.3.1 指令系统设计的基本原则

4. 在设计指令系统时，有两种截然不同的设计策略。

（产生了两类不同的计算机系统：CISC和RISC）

- ◆ RISC（精简指令系统计算机）：追求单周期、低CPI、流水线
 - ✓ 减少CPI，尽量使用单周期指令
 - ✓ 尽可能减少时钟周期时间
 - ✓ 尽可能地把指令系统简化，不仅指令的条数少，而且指令的功能也比较简单。
 - ✓ 优化编译，有效支持高级语言
- ◆ 实现方式：主要依靠硬件实现，增加了通用寄存器，以硬布线逻辑控制为主；适合流水线

以下关于复杂指令集计算机（Complex Instruction Set Computer,CISC）的叙述中正确的是（）

- ☐ A 只设置使用频度高的一些简单指令，不同指令执行时间差别很小
- ☐ B CPU 中设置大量寄存器，利用率低
- ☐ C 常采用执行速度更快的组合逻辑实现控制器
- ☒ D 指令长度不固定，指令格式和寻址方式多

提交



➤ **控制指令**是用来改变**控制流**的。

- ◆ **跳转**：当指令是**无条件改变控制流**时，称之为**跳转指令**。
- ◆ **分支**：当控制指令是**有条件改变控制流**时，则称之为**分支指令**。

➤ **能够改变控制流的指令**

- ✓ **分支**
- ✓ **跳转**
- ✓ **过程调用（特殊跳转）**
- ✓ **过程返回（特殊跳转）**

控制指令的使用频度

（load-store型指令系统结构的机器，基准程序为SPEC CPU2000）

指令类型	使用频度	
	整型平均	浮点平均
调用/返回	19%	8%
跳转	6%	10%
分支	75%	82%

改变控制流的大部分指令是**分支指令（条件转移）**

联系：？指令系统设计：以经常使用事件为重点



2.3 指令系统的设计与优化

2.3.2 控制指令

1、分支条件的方法及其优缺点

名 称	检测分支条件的方法	优 点	缺 点
条件码 (CC)	检测由ALU操作设置的一些特殊的位（即CC），例如Z、N、V、C	可以自由设置分支条件	条件码是增设的状态。而且它限制了指令的执行顺序，因为要保证条件码能顺利地传送给分支指令（即不能随便插入那些会修改条件码的指令）
条件寄存器	比较指令把比较结果放入任何一个寄存器，检测时就检测该寄存器 例如BNE R3,label1	简单	占用了一个寄存器
比较与分支	比较操作是分支指令的一部分，通常这种比较是受到一定限制的 BEQ R1,R2,offset，一条指令内部完成 R1 与 R2 比较，如果相等就跳转。	用一条指令（而不是两条）就能实现分支	当采用流水方式时，该指令的操作可能太多，在一拍内做不完（又要比较又要跳转）。



4、控制流指令的寻址方式（（跳转、分支、调用、返回）怎么找到要跳过去的目标地址？）

- ◆ 需要指明转移的目标地址，多数情况下，目标地址会在指令中明确表示出来
- ◆ 过程返回指令（例如 RET）是例外，因为编译时不知道返回的地址（当多调用产生时），所以不能写在指令里（只能运行时保存，例如保存栈或寄存器中）
- ◆ 最常见的是使用PC的位移量来指定目标地址，这种类型的控制流指令为PC相对寻址指令

BEQ R1, R2, offset

如果 $R1 == R2$ ，则 $PC = PC + offset$



5. PC转移目标地址的表示（这样表示的好处）

- ◆ 最常用的方法：在指令中提供一个偏移量，由该偏移量和程序计数器（PC）的值相加而得出目标地址。（PC相对寻址）
- ◆ 优点
 - ✓ 有效地减少表示该目标地址所需要的位数（否则要写出完整的地址位数），缩短指令的长度
 - ✓ 使程序的运行与载入位置无关（称为位置无关）（代码可被装载到主存的任意位置执行），可以减少程序在链接时的工作量，对在执行时才链接的程序有好处（优化，只考虑不变化的相对距离即可）。
- ◆ 关键：确定偏移量字段的长度---跳转的目标地址离跳转指令有多远
 - ✓ 模拟结果表明：采用4~8位的偏移量字段（以指令字为单位）就能表示大多数控制指令的转移目标地址了（否则不用偏移量字段而采用寄存器间接寻址即可扩大范围）。



6. 过程调用和返回（控制流指令中最复杂的一类）

- ◆ 修改 PC，切换控制流，跳转到子程序入口（例如 call function1）；
- ◆ 除了要改变控制流之外，可能还要保存机器状态。至少也得保存返回地址（call指令之后的指令地址，放在专用的链接寄存器或堆栈中）
 - ✓ 编译时不知道目标地址，为了实现返回和间接跳转，需要使用寄存器间接跳转（动态指定目标地址，才能在运行时改变目标地址）
- ◆ 过去有些指令系统结构提供了专门的保存机制来保存许多寄存器的内容。
- ◆ 现在较新的指令系统结构则要求由编译器生成 load 和 store 指令来保存或恢复寄存器的内容。



2.3 指令系统的设计与优化

2.3.3 指令操作码的优化

- ◆ 指令由两部分组成：操作码、地址码
- ◆ 指令格式的设计：确定指令字的编码方式，包括操作码字段和地址码字段的编码和表示方式。
- ◆ 指令格式的优化：如何用最短的位数来表示指令的操作信息和地址信息。
- ◆ 定长操作码优化没有空间，只有不定长的操作码才可优化。
- ◆ 优化的目的：提高性能，降低消耗（空间）
 - ✓ ——两者都希望 操作码变短（优化的思路，从性能和空间）

先定原则，再抓重点（控制指令，尤其是分支），最后在编码层面优化（操作码）。



2.3 指令系统的设计与优化

2.3.3 指令操作码的优化

- 对操作码优化：首先需要知道每种指令在程序中出现的概率（使用频度），可通过大量经典程序统计求得。
- 哈夫曼编码（变长操作码优化）
 - ◆ **基本思想**：当各种事件发生的概率不均等时，可以对发生概率最高的事件用最短的位数（时间）来表示（处理），而对于出现概率较低的事件，则可以用较长的位数（时间）来表示（处理），从而使总的平均位数（时间）缩短。
 - ◆ 构造哈夫曼树的方法
 - ✓ 上述基本思想可以借助哈夫曼编码实现

常用的指令，编码短一点；不常用的指令，编码长一点；**并且：平均长度最短**



2.3.3 指令操作码的优化

➤ 操作码优化的程度可以用信息熵来衡量（优化的理论，理论上的最优）。

□ 信息熵：即信息所含的平均信息量。理论最短平均码长

$$H = - \sum_{i=1}^n p_i \log_2 p_i$$

- ◆ 表示用二进制编码表示 n 个码点（指令）时，理论上的最短平均编码长度。
- ◆ P_i ，第 i 个事件发生的概率（这里就是第 i 条指令的使用频率）
- ◆ 每个事件的信息量（和某条指令的有关的对数值）： $-\log_2 p_i$
- ◆ 加权平均： H

在操作码编码中：信息熵 H 就是--在已知每条指令使用频率的情况下，操作码平均最少需要多少位来编码。



2.3 指令系统的设计与优化

2.3.3 指令操作码的优化

例2.1 假设某模型机有7条指令，这些指令的使用频度如表左边所示。

- (1) 计算这7条指令的操作码编码的最短平均码长；
- (2) 画出哈夫曼树，写出这7条指令的哈夫曼编码，并计算该编码的平均码长和信息冗余量。

指令	频度 p_i	操作码使用哈夫曼编码	操作码长度 l_i	利用哈夫曼概念的扩展操作码	操作码长度 l_i
I_1	0.40	0	1	0 0	2
I_2	0.30	1 0	2	0 1	2
I_3	0.15	1 1 0	3	1 0	2
I_4	0.05	1 1 1 0 0	5	1 1 0 0	4
I_5	0.04	1 1 1 0 1	5	1 1 0 1	4
I_6	0.03	1 1 1 1 0	5	1 1 1 0	4
I_7	0.03	1 1 1 1 1	5	1 1 1 1	4



2.3 指令系统的设计与优化

2.3.3 指令操作码的优化

解 (1) 这7条指令的操作码编码的最短平均码长;

不管怎么编码, 平均码长都不可能低于 2.17 位

$$H = - \sum_{i=1}^7 p_i \log_2 p_i = 2.17$$

(2) 其哈夫曼树如图所示, 该树的每个叶结点分别对应于一条指令。在该树中, 对每个结点向下的两个分支, 分别用二进制“1”和“0”来表示。

从该哈夫曼树可以很容易地写出哈夫曼编码。

具体方法: 对于任意一条指令 I_i ($i=1, 2, \dots, 7$), 从哈夫曼树根结点出发、沿一条路径连接到叶结点 I_i , 把途中所经过的各分支的“0”和“1”按从左到右的顺序记录下来, 便是该指令的哈夫曼编码。上表中列出了所有指令的哈夫曼编码。



2.3 指令系统的设计与优化

2.3.3 指令操作码的优化

解 (1) 这7条指令的操作码若用定长编码：需要3位，

这7条指令的操作码编码的最短平均码长为：

$$H = -\sum_{i=1}^7 p_i \log_2 p_i = 2.17$$

$$\begin{aligned} H &= 0.40 \times 1.32 + 0.30 \times 1.74 + 0.15 \times 2.74 + 0.05 \times 4.32 + \\ &\quad 0.04 \times 4.64 + 0.03 \times 5.06 + 0.03 \times 5.06 \\ &= 2.17 \end{aligned}$$

说明：表示这7种指令，操作码平均需要2.17位就足够了



2.3 指令系统的设计与优化

- **信息冗余量：**信息冗余量是指在信息传输或存储过程中，由于某些字符或信息的重复出现而产生的多余数据。（可以理解为浪费了多少？）
- 结合本节案例，定长操作码编码信息冗余量（高频可以用短码，结果却用了和低频指令一样的编码，冗余了指令）的计算如下：

1、
$$\frac{\text{操作码的平均码长} - H}{\text{操作码的平均码长}}$$

$$\frac{3 - 2.17}{3} \approx 28\%$$



说明：采用实际定长3位操作码编码，浪费了28%



优化：哈夫曼编码



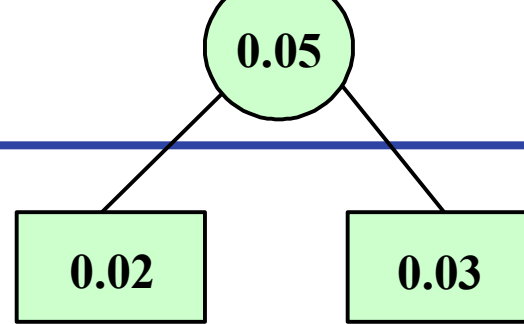
2.3 指令系统的设计与优化

2.3.3 指令操作码的优化

1. 哈夫曼编码

➤ 构造哈夫曼树的方法

- ✓ 将各事件按其使用频度从小到大依次排列；
- ✓ 每次从中选择两个频度值最小的结点，将其合并成一个新的结点，并把新结点画在所选结点的上面，
- ✓ 然后用两条边把新结点分别与那两个结点相连。
 - 新结点的频度值是所选两个结点的频度值的和。
- ✓ 把新结点与其他剩余未结合的结点一起，再以上面的步骤进行处理，反复进行，直到全部结点都结合完毕、形成根结点为止。



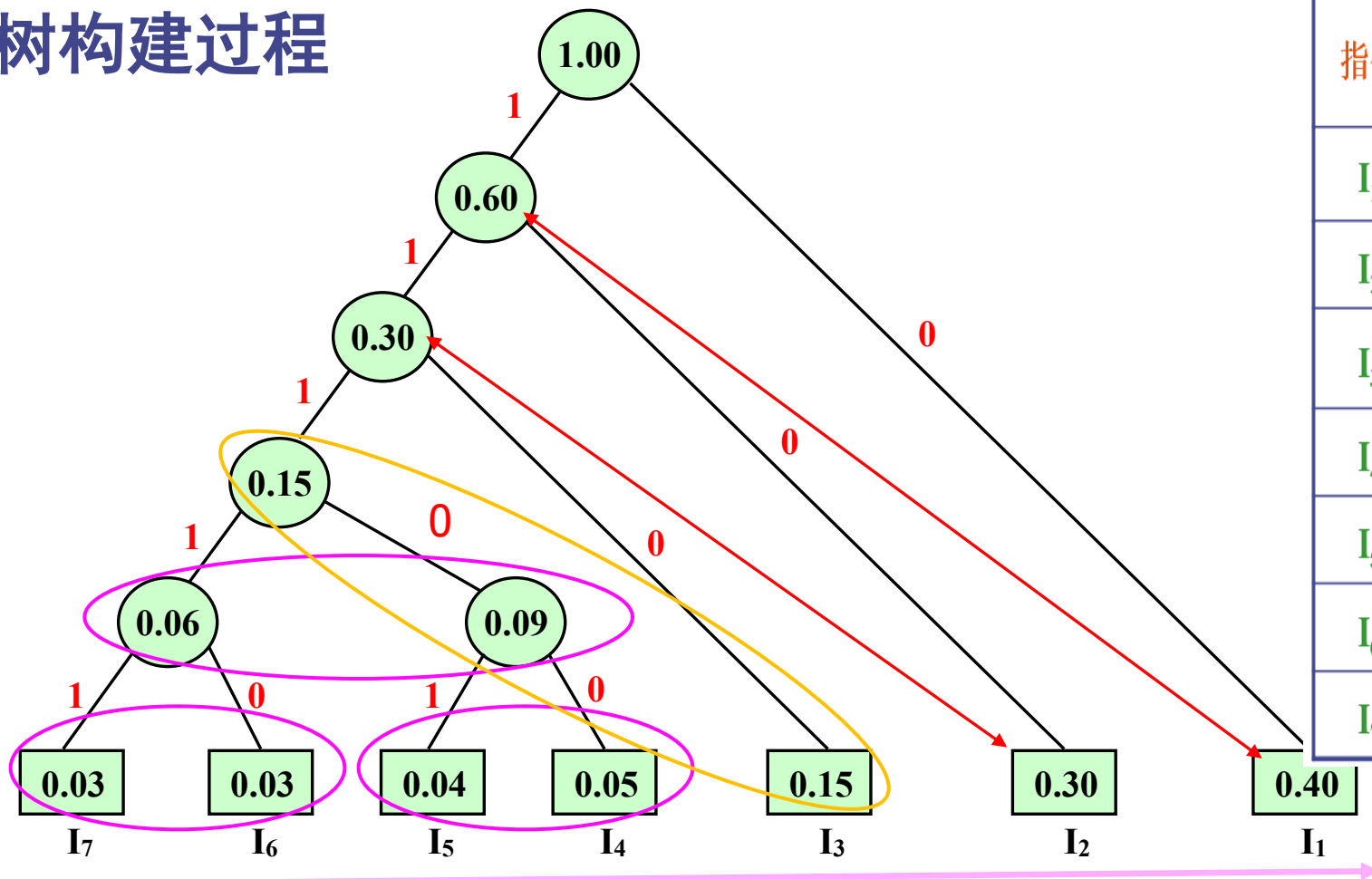
画哈夫曼树的一个基本步骤



2.3 指令系统的设计与优化

哈夫曼树构建过程

指令	频度 p_i
I_1	0.40
I_2	0.30
I_3	0.15
I_4	0.05
I_5	0.04
I_6	0.03
I_7	0.03



指令	频度 p_i	操作码使用 哈夫曼编码
I_1	0.40	0
I_2	0.30	10
I_3	0.15	110
I_4	0.05	11100
I_5	0.04	11101
I_6	0.03	11110
I_7	0.03	11111



2.3 指令系统的设计与优化

- 根据上页采用哈夫曼编码，该哈夫曼编码的平均码长（平均码长是唯一的）：

指令	频度 p_i	操作码使用哈夫曼编码
I_1	0.40	0
I_2	0.30	10
I_3	0.15	110
I_4	0.05	11100
I_5	0.04	11101
I_6	0.03	11110
I_7	0.03	11111

$$L = 0.40 \times 1 + 0.30 \times 2 + 0.15 \times 3 + 0.05 \times 5 + 0.04 \times 5 + 0.03 \times 5 + 0.03 \times 5 = 2.20$$

$$L = \sum_{i=1}^7 p_i l_i = 2.20$$

l_i 为第*i*条指令的哈夫曼编码的长度

- 这7条指令的定长操作码编码的最短平均码长：

$$H = -\sum_{i=1}^7 p_i \log_2 p_i = 2.17$$

- 这7条指令的哈夫曼编码 平均码长： $L = \sum_{i=1}^7 p_i l_i = 2.20$



2.3 指令系统的设计与优化

- **信息冗余量：**信息冗余量是指在信息传输或存储过程中，由于某些字符或信息的重复出现而产生的多余数据。
- **结合本节案例，两种信息冗余量的计算如下：**

1、
$$\frac{\text{操作码的平均码长} - H}{\text{操作码的平均码长}} = \frac{2.20 - 2.17}{2.20} \approx 1.36\%$$



说明：采用哈夫曼编码非常接近最短长度2.17，浪费了1.36%

2、
$$\frac{\text{操作码的实际平均码长} - H}{\text{操作码的实际平均码长}} = \frac{3 - 2.17}{3} \approx 28\%$$



说明：采用实际定长3位操作码编码，浪费了28%



2.3 指令系统的设计与优化

➤ 根据上页采用哈夫曼编码，该哈夫曼编码的平均码长：

$$L = 0.40 \times 1 + 0.30 \times 2 + 0.15 \times 3 + 0.05 \times 5 + 0.04 \times 5 + 0.03 \times 5 + 0.03 \times 5 = 2.20$$

$$L = \sum_{i=1}^7 p_i l_i = 2.20$$

指令	频度 p_i	操作码使用哈夫曼编码	操作码长度 l_i	利用哈夫曼概念的扩展操作码	操作码长度 l_i
I_1	0.40	0	1	0 0	2
I_2	0.30	1 0	2	0 1	2
I_3	0.15	1 1 0	3	1 0	2
I_4	0.05	1 1 1 0 0	5	1 1 0 0	4
I_5	0.04	1 1 1 0 1	5	1 1 0 1	4
I_6	0.03	1 1 1 1 0	5	1 1 1 0	4
I_7	0.03	1 1 1 1 1	5	1 1 1 1	4

长度不唯一、
变长，会带来
什么影响

操作码规整性
太差



2.3 指令系统的设计与优化

➤ 根据上页引入扩展操作码：

操作码扩展

这三条指令出现的
频度：共
0.85

这四条指令出现的
频度：共
0.15

指令	频度 p_i	操作码使用 哈夫曼编码	操作码 长度 l_i	利用哈夫曼概念 的扩展操作码	操作码 长度 l_i
I_1	0.40	0	1	0 0	2
I_2	0.30	1 0	2	0 1	2
I_3	0.15	1 1 0	3	1 0	2
I_4	0.05	1 1 1 0 0	5	1 1 0 0	4
I_5	0.04	1 1 1 0 1	5	1 1 0 1	4
I_6	0.03	1 1 1 1 0	5	1 1 1 0	4
I_7	0.03	1 1 1 1 1	5	1 1 1 1	4

高频短码
低频长码

便于识别：
将11分配给
4位编码

1-2-3-5编码

2-4编码



2.3.3 指令操作码的优化

➤ 采用扩展编码，只保留两种2、4两种长度，操作码长度如下：

✓ 用两位的00、01、10分别用于表示使用频度高的 l_1 、 l_2 、 l_3 ，然后用11作为高位扩展出4个4位的二进制编码，用于表示剩下的4条指令。

✓ 平均长度

$$L = \sum_{i=1}^7 p_i l_i = 2.3$$

$$L = 0.40 \times 2 + 0.30 \times 2 + 0.15 \times 2 + 0.05 \times 4 + 0.04 \times 4 + 0.03 \times 4 + 0.03 \times 4 \\ = 2.30$$

比哈夫曼编码的2.2大，但很接近于2.2，而且比定长的3位编码 小很多。



2.3.3 指令操作码的优化

3、信息冗余量

$$\frac{\text{操作码的平均码长} - H}{\text{操作码的平均码长}} \longrightarrow \frac{2.30 - 2.17}{2.30} \approx 5.56\%$$

$$-\sum_{i=1}^7 p_i l_i = 2.3$$

□ 扩展编码平均长度

说明：采用扩展操作码编码，浪费了5.56%

比较三种冗余量：种类个数、长度大小、规整等方面

定长编码（3位）：平均码长 = 3.00 位、纯哈夫曼编码：平均码长 $\approx$ 2.20 位

2-4扩展编码：平均码长 = 2.30 位

结论：扩展编码比定长编码节省了0.7位/指令，性能提升约23%；虽然比纯哈夫曼编码略长0.1位，但硬件实现难度大幅降低。



2.3.3 指令操作码的优化

➤ 一个指令系统中

- ◆ 若操作码长度固定且指令格式不同，指令格式如右：

OP	A1	A2	A3
OP	A1	A2	无用
OP	A1	无用	无用
OP	无用	无用	无用

- ◆ 操作码字段长度取决于指令系统中的指令总数目；
- ◆ 地址码较少的指令，编码浪费；

➤ 操作码扩展(扩展方式较多)

- ◆ 对于地址码较少的指令，把它们的操作码扩充到不用的地址码字段；
- ◆ 既充分利用指令字的各字段，又在不增加指令长度的情况下扩展操作码的长度。
- ◆ 例如哈夫曼扩展编码



2.3.3 指令操作码的优化

现有问题需求

- 既能像哈夫曼那样让常用指令短一点；
- 又要让编码有规律，硬件能快速分级译码。

扩展操作码

- 位于定长二进制编码和哈夫曼编码之间的一种编码方案。
- 采用有限几种固定长度的码长（例如例2.1表中2、4），仍然采用高概率的用短码、低概率用长码的哈夫曼压缩思想，使操作码平均长度缩短。---尽量规整（上表例2.1）
- 如例2.1表中的指令，采用2-4的扩展操作码，可以得到如表右边所示的编码方案。

指令	频度 p_i	操作码使用哈夫曼编码	操作码长度 l_i	利用哈夫曼概念的扩展操作码	操作码长度 l_i
I_1	0.40	0	1	0 0	2
I_2	0.30	1 0	2	0 1	2
I_3	0.15	1 1 0	3	1 0	2
I_4	0.05	1 1 1 0 0	5	1 1 0 0	4
I_5	0.04	1 1 1 0 1	5	1 1 0 1	4
I_6	0.03	1 1 1 1 0	5	1 1 1 0	4
I_7	0.03	1 1 1 1 1	5	1 1 1 1	4



2.3.3 指令操作码的优化

“分级译码”：先看前4位，不够再看后4位。

2. 扩展操作码---等长扩展码

- ◆ 为了便于分级译码，一般都采用**等长扩展码**，即展开结点的层级数之间是等差数列。
- ◆ **等长**：每次扩展的**位数相等**，便于实现分级译码。例如操作码按4位、8位、12位码长，每次**加长4位**扩展方法，称作4-8-12扩展法。

例如：**15/15/15**法和**8/64/512**法（都是4-8-12扩展法）

✓ 每次扩展4位编码

- ◆ 假设指令字长**16位**，最多三地址指令（分析15/15/15编码方法）

若 操作码**4位**：16种状态；操作码**8位**：256种状态；操作码**12位**：4096种状态

操作码4位	地址1—4位	地址2---4位	地址3---4位
-------	--------	----------	----------



2.3.3 指令操作码的优化

2. 扩展操作码----等长扩展码

“分级译码”：先看前 4 位，不够再看后 4 位。

◆ 假设指令字长16位，最多三地址指令（分析15/15/15编码方法）

若 操作码4位：16种状态；操作码8位：256种状态；操作码12位：4096种状态

操作码4位	地址1—4位	地址2---4位	地址3---4位
-------	--------	----------	----------

15/15/15 编码方法 分配方案

指令类型	操作码位数	指令条数	操作码编码范围
三地址指令	4 位	15 条（留4位操作码1111 做扩展标志）	0000 ~ 1110
二地址指令	8 位	15 条 同上，留1111 1111	1111 0000 ~ 1111 1110
一地址指令	12 位	15 条 同上，留1111 1111 1111	1111 1111 0000 ~ 1111 1111 1110
零地址指令	16 位	16 条	1111 1111 1111 0000 ~ 1111 1111 1111 1111

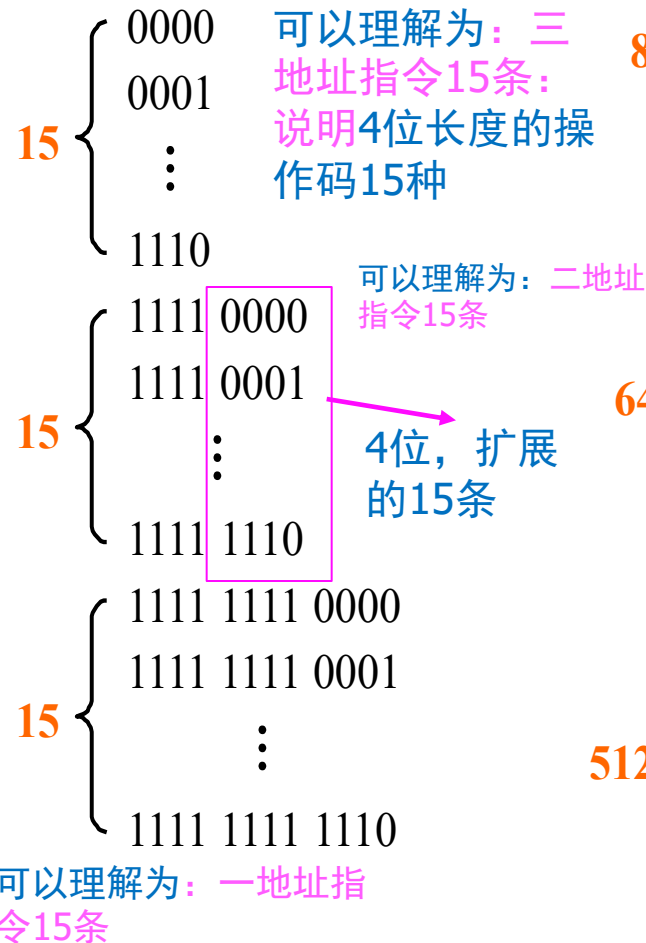
每条指令都留 1 个编码用于扩展，所以每一级都是 15 条。----因此称为 15/15/15编码方法



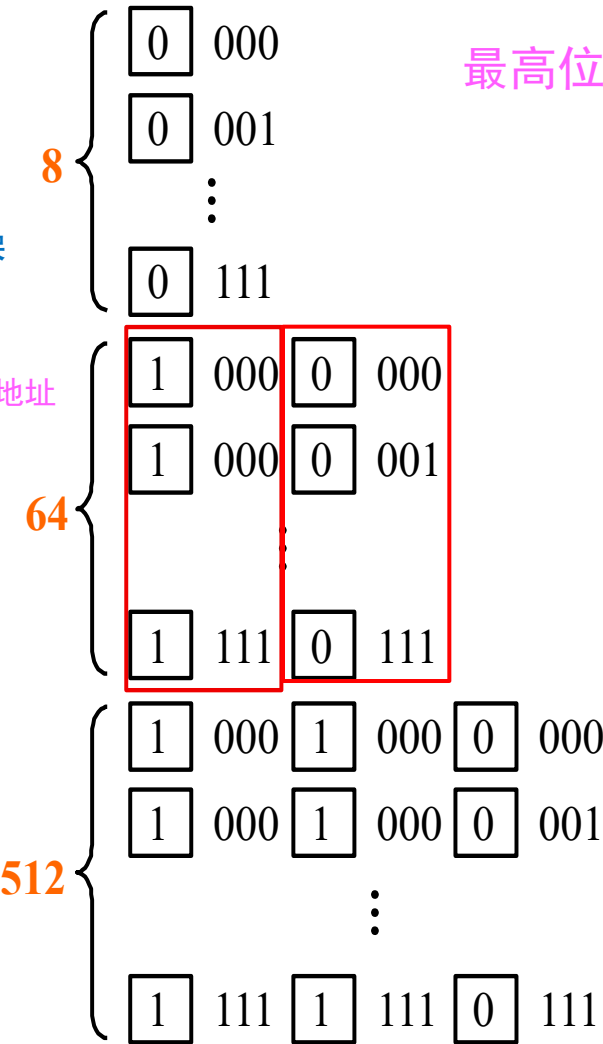
2.3.3 指令操作码的优化

操作码等长扩展

15/15/15 编码法：
每一级的指令条数
都是15条

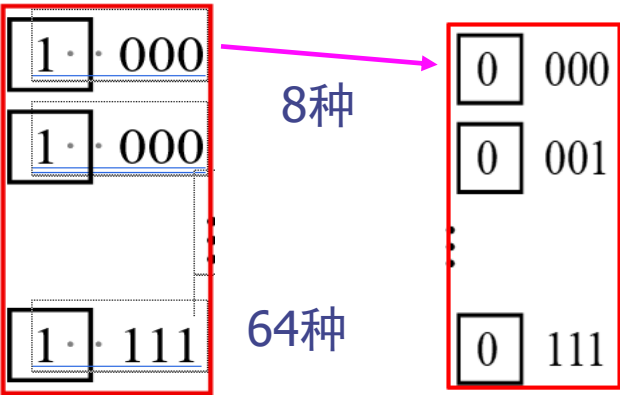


15/15/15 编码法



8/64/512 编码法

最高位0不变，低三位扩展8种



到底哪种优化好？（扩展条数的变化规律）
视场景使用而定

8/64/512 法：根据需要调整每一级的指令条数



2.3.3 指令操作码的优化

操作码编码	说 明
0000 0001 1110	4 位长度的 操作码共 15 种
1111 0000 1111 0001 1111 1110	8 位长度的 操作码共 15 种
1111 1111 0000 1111 1111 0001 1111 1111 1110	12 位长度的 操作码共 15 种

等长 15/15/15.....扩展法

操作码编码	说 明
0000 0001 0111	4 位长度的 操作码共 8 种
1000 0000 1000 0001 1111 0111	8 位长度的 操作码共 64 种
1000 1000 0000 1000 1000 0001 1111 1111 0111	12 位长度的操 作码共 512 种

等长 8/64/512.....扩展法



2.3.3 指令操作码的优化

2. 扩展操作码----等长扩展码

◆ 例如：15/15/15法和8/64/512法（都是4-8-12扩展法）

- ✓ 选用哪种编码法取决于指令使用频度 p_i 的分布、对应地址的指令条数，然后计算其操作码平均码长，然后确定选择15/15/15法还是选择8/64/512法。
- ✓ 衡量标准：看哪种编码法能使平均码长度最短。



2.3.4 指令字格式的优化

指令格式优化：把操作码和地址码放在一起统筹规划，让整条指令更短、更规整、更好译码。

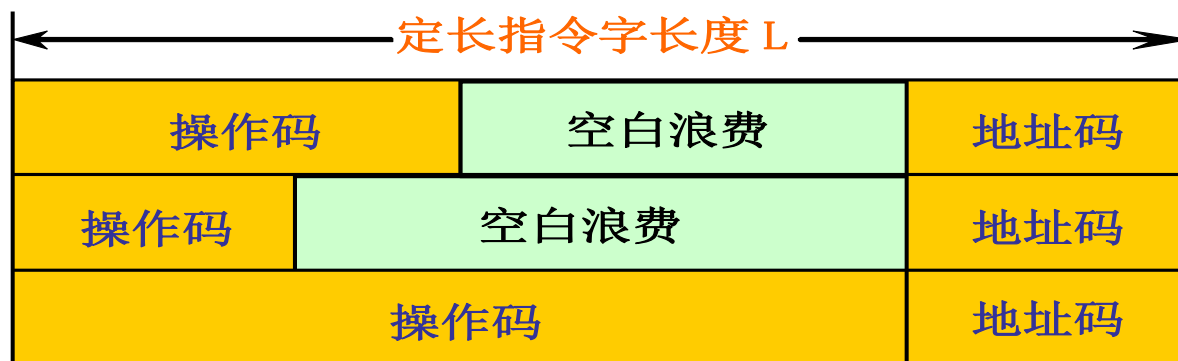
➤ 指令字格式的优化措施

- ✓ 与操作码优化配合，采用扩展操作码方式（解决了操作码优化，但地址码也变化了怎么办）
- ✓ 整数边界对齐等（硬件取指方便、便于流水、访存效率高等）
- ✓ 指令格式中，操作码定长情况下对地址码的优化（定长操作码没有优化空间，但地址码可以优化，见下两页）
- ✓ 操作数寻址方式的优化（减少寻址种类、隐含寻址等—即让 操作数寻址 占较少的位数）
- ✓ 。 。 。



2.3.4 指令字格式的优化

1. 若指令字的宽度固定，地址码的长度和个数固定，则操作码的缩短并不能带来好处，只是使指令字中出现空白浪费。

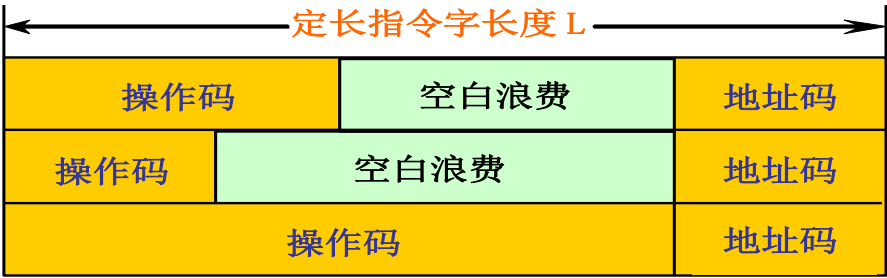


根据哈夫曼编码，频度高操作码长度短，反之长；如果指令字的宽度固定，地址码的长度和个数固定，则操作码的缩短使指令字中出现空白浪费。（怎么办）



2.3.4 指令字格式的优化

1. 若指令字的宽度固定，地址码的长度和个数固定，则操作码的缩短，指令字中出现空白浪费。



操作码缩短浪费的空间如何优化（两个方案）：采用地址个数可变/地址码长度可变的方案

➤ 最常用的操作码最短，其地址字段个数最多。能够使指令的功能增强，从总体上减少所需的指令条数。

a、定长指令字下采用多种地址方法

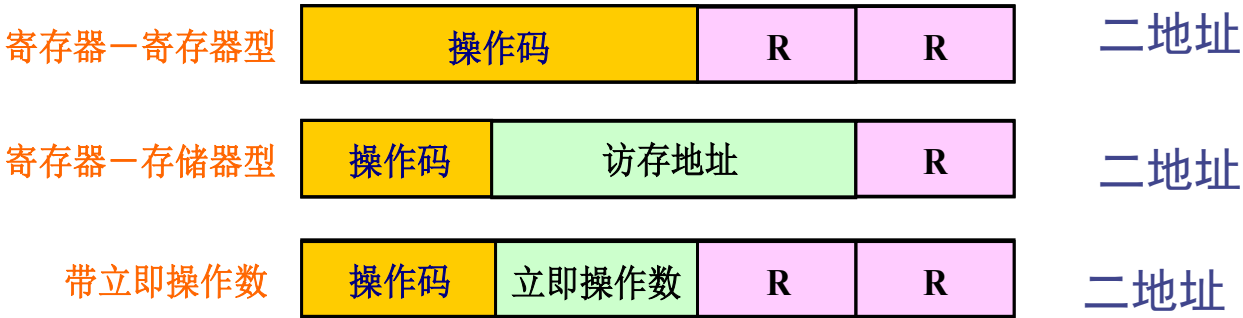


一地址

二地址

三地址

b、同种地址下多种地址形式和变化





2.3.4 指令字格式的优化

2、在变长指令字中，适合当寻址方式和操作数种类很多的情况下，采用操作码扩展与可变长地址码/寻址方式字段相结合的指令格式



3、混合型编码格式，适合需要目标代码长度段，且能降低译码复杂度的需求。此时可采用上述两种方式混合编码

4、指令格式优化考虑因素

- ◆ 指令字长度---程序平均长度
- ◆ 在指令系统的设计中，要在指令字长与寄存器的个数以及寻址方式的个数之间进行折中。
- ◆ 计算机中寄存器的个数和寻址方式的数目对机器的指令字长的影响；



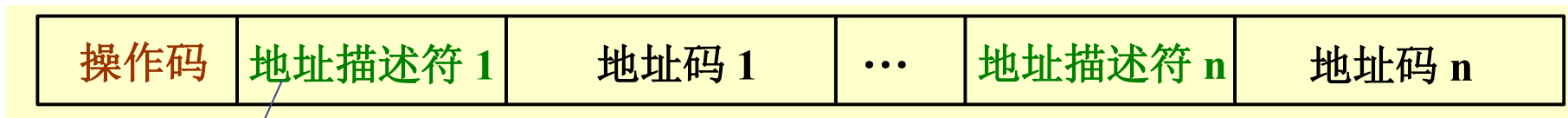
2.3.4 指令字格式的优化

➤ 5、指令系统的3种编码格式

◆ 可变长度编码格式、固定长度编码格式、混合型编码格式

➤ 可变长度编码格式（侧重代码长度的选择）-侧重CISC

- ◆ 当指令系统的寻址方式和操作种类很多时，允许所有的操作使用所有的寻址方式，这种编码格式是最好的。
- ◆ 用最少的二进制位来表示目标代码，代码量最小。（例如高频短码、低频长码）
- ◆ 可能会使各条指令的字长和执行时间相差很大。（指令字长不同）



告诉CPU后面地址码1是存储器地址还是寄存器地址



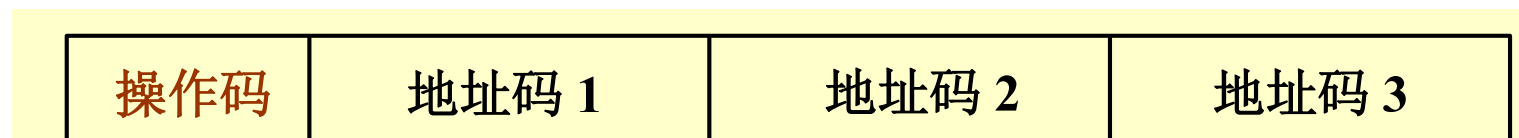
2.3.4 指令字格式的优化

➤ 固定长度编码格式（侧重性能的选择）--侧重RISC

- ◆ 操作类型和寻址方式一起编码到操作码中，通常所有的指令长度都相同。
- ◆ 当寻址方式和操作类型非常少时，这种编码格式非常好。
- ◆ 可以有效地降低译码的复杂度，提高译码的速度。
- ◆ 大部分RISC的指令系统均采用这种编码格式。
- ◆ 但代码量大

➤ 固定长度编码格式--好处

- ◆ 若操作码短（例如使用频度高的指令），地址个数可变或地址码长度可变---指令功能增强，减少指令条数。





2.3 指令系统的设计与优化

➤ 混合型编码格式（结合上述两种方案）

- ◆ 提供若干种固定的指令字长,多种指令长度以减少代码长度。
- ◆ 以期达到既能够减少目标代码长度又能降低译码复杂度的目标。





2.3 指令系统的设计与优化

➤ 可变长度编码和固定长度编码的区别

- ◆ 两者之间的取舍实质上是程序大小和处理器译码难度之间的取舍
- ◆ 变长编码总是试图用最少的位数来表示程序，但是每条指令在指令长度机器可操作上有很大的差别。

➤ 指令系统编码总结

- ◆ 关注生成代码量的大小---选择变长编码方法（代码量最小）。
- ◆ 关注程序运行性能 ----选择定长编码方法。



2.4 指令系统的发展和改进

2.4.1 沿CISC方向发展和改进指令系统 (关注功能)

2.4.1 沿RISC方向发展和改进指令系统 (关注速度)



2.4 指令系统的发展和改进

2.4.1 沿CISC方向发展和改进指令系统

➤ CISC指令系统的一大特点

指令数量多、功能多样 （可变长度编码格式的适应场景）

主要是从以下3个方面着手改进：

- （1）面向目标程序增强指令功能（例如：编译器生成的代码）
- （2）面向高级语言的优化实现来改进指令系统（例如：程序员关注的）
- （3）面向操作系统的优化实现改进指令系统（例如：底层系统管理员关注）

➤ （1）面向目标程序增强指令功能

- ✓ 提高系统性能最直接的方法（CPU直接执行的目标程序—机器指令）
- ✓ 目的：既能减少目标程序的执行时间，也能有效地缩短程序的长度。



2.4.1 沿CISC方向发展和改进指令系统

➤ (1) 面向目标程序增强指令功能

◆ 途径：根据执行情况统计，对频度进行分析（例如操作优化），分析哪些功能仍用基本指令串实现，哪些功能可改用新指令，哪些可提高机器语言目标程序的实现效率等

✓ 使用频度高的指令：硬件实现加快其执行

✓ 高频使用的指令串：用一条指令来实现

既能减少目标程序的执行时间，又能有效缩短程序的长度

➤ 可以从以下3个方面来改进：

- ◆ 增强运算型指令的功能（利用微程序实现代替宏指令或子程序），再例如一条多项式指令实现 $a*b+c$
- ◆ 增强数据传送指令的功能（高频指令串合并），例如实现成组传输，如 **cache** 的使用
- ◆ 增强程序控制指令的功能，例如设置多种条件下的转移指令和子程序控制指令



2.4 指令系统的发展和改进

其中：增强程序控制指令的功能：

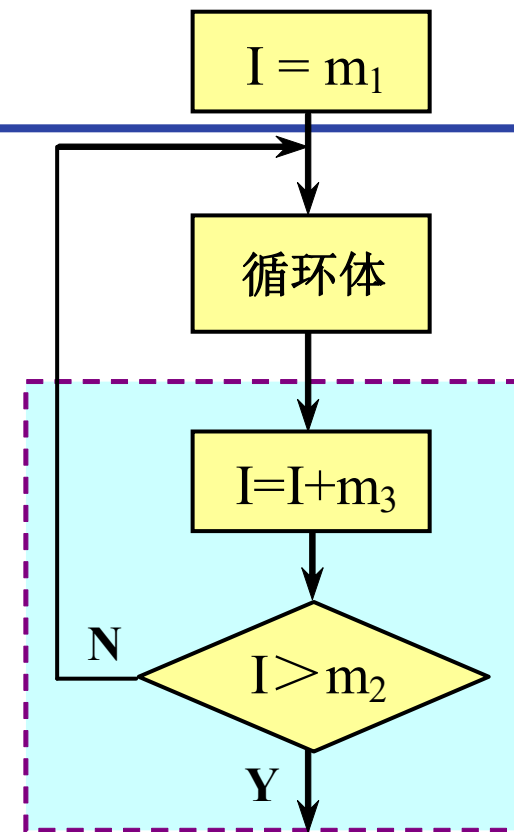
例如：循环在程序中占有相当大的比例，所以在指令上提供专门的支持。

➤ 循环控制部分通常用3条指令完成：

- ✓ 一条加法指令 ($I = I + m_3$)
- ✓ 一条比较指令 (例如大于)
- ✓ 一条分支指令

➤ 设置循环控制指令（例如 LOOP 指令），用一条指令完成 一般循环程序的结构
上述3条指令的功能。

上述三个方面对改进目标程序已获得较好的成果，
但增加了硬件的成本及复杂度





2.4 指令系统的发展和改进

➤ (2) 面向高级语言的优化实现来改进指令系统

◆ **目标：** 缩小高级语言与机器语言的语义差距，缩短编译程序长度和编译时间

◆ **实现途径**

例如高级语言 $a=c-1$ ，机器指令需要多条指令完成

✓ 增强对高级语言和编译器的支持

- 对高级语言中使用频度高、执行时间长的语句，增强有关指令的功能，加快这些指令的执行速度（例如函数调用、数组访问最耗时，增强相应的寻址方式，一步到位）
- 一元赋值语句、条件转移和无条件转移语句占比比较大——优化
- 或者增加专门的指令，可以达到减少目标程序的执行时间和减少目标程序长度的目的。例如科学运算中的矩阵、向量处理等（一条指令同时处理多个数据）
- 增强系统结构的规整性，减少系统结构中的各种例外情况（例如让所有运算指令不仅仅能访问寄存器也可以直接访问内容，即访问格式统一）



2.4 指令系统的发展和改进

➤ (2) 面向高级语言的优化实现来改进指令系统

◆ 其他途径：面向高级语言的计算机

- ✓ 间接执行高级语言机器（高级语言称为机器的汇编语言，二者一一高度对应，编译则会工作简单，语义差距小，例如高级语言矩阵加法 $a=b+c$ ）
- ✓ 直接执行高级语言的机器--直接把高级语言作为机器语言，直接用固件或硬件对高级语言程序的语句逐条进行解释执行，这时既不用编辑也不用汇编（但这可能吗—太激进，结构太复杂等）

CISC困境：极力做好各种优化，可能进入两个极端，希望计算机优化性能、尽量能做到各种支持，这时我们的计算机称为上述两种机器



2.4 指令系统的发展和改进

➤ (3) 面向操作系统的优化实现改进指令系统

- ✓ 操作系统和计算机系统结构是紧密联系的，操作系统的实现在很大程度上取决于系统结构的支持。
- ✓ 指令系统对操作系统的支持主要有：
 - 处理机工作状态和访问方式的切换；--硬件状态位
 - 进程的管理和切换；---现场保护
 - 存储管理和信息保护；---多用户共用存储器
 - 进程的同步与互斥，信号灯的管理等。

上述这些支持操作系统的有些指令属于特权指令，一般用户程序是不能使用的。

- 途径：改进指令系统，使它与各种语言间的语义差距都有同等的缩小，减少OS辅助操作时间，节省OS占用空间



2.4 指令系统的发展和改进

2.4.2 沿RISC方向发展和改进指令系统

➤ CISC指令系统结构存在的问题

各种指令的使用频度相差悬殊，许多指令很少用；

- ✓ 据统计：只有20%的指令使用频度比较高，占运行时间的80%。而其余80%的指令只在20%的运行时间内才会用到。
- ✓ 使用频度高的指令也是最简单的指令。指令系统庞大，指令条数很多，许多指令的功能又很复杂，使得控制器硬件非常复杂。
- ✓ 许多指令由于操作繁杂，其CPI值比较大，执行速度慢。采用这些复杂指令有可能使整个程序的执行时间反而增加。
- ✓ 由于指令功能复杂，规整性不好，不利于采用流水技术来提高性能。



2.4 指令系统的发展和改进

2. 设计RISC机器遵循的原则

- ◆ 指令条数少、指令功能简单。只选取使用频度很高的指令，在此基础上补充一些最有用的指令；
- ◆ 采用简单而又统一的指令格式，并减少寻址方式；指令字长都为32位或64位；
- ◆ 指令的执行在单个机器周期内完成；（采用流水线后，理想情况下每个周期都能完成一条指令（吞吐率为1））
- ◆ 只有load和store指令才能访问存储器，其它指令的操作都是在寄存器之间进行；（即采用load-store结构）
- ◆ 大多数指令都采用硬连逻辑来实现；



2.4 指令系统的发展和改进

➤ 设计RISC机器遵循的原则

- ◆ 强调优化编译器的作用，为高级语言程序生成优化的代码；
- ◆ 充分利用流水技术来提高性能。

3. 早期的RISC微处理器

- ◆ 1981年，Berkeley分校的Patterson 等人的32位微处理器RISC I：
 - ✓ 31条指令，指令字长都是32位，78个通用寄存器，时钟频率为8MHz；
 - ✓ 控制部分所占的芯片面积只有约6%。商品化微处理器MC68000和Z8000分别为50%和53%；
 - ✓ 性能比MC68000和Z8000快3~4倍。



2.4 指令系统的发展和改进

◆ 1981年，Stanford大学Hennessy等人的MIPS

✓ 后来发展成了MIPS Rxxx系列微处理器。

◆ 1983年的RISC II:

✓ 指令条数为39，通用寄存器个数为138，时钟频率为12MHz。

✓ 后来，1987年发展成了Sun公司的SPARC系列微处理器。

◆ 1992年DEC公司转为高端台式机、工作站服务器设计的微处理器架构Alpha问世

◆ MIPS风靡一时，2011年龙芯中科购买Mips架构授权—继续优化

◆ IBM的801

共同特点：采用load-store结构；指令字长为32位（或64位）；采用高效的流水技术



2.4 指令系统的发展和改进

➤ RISC技术优势

- ◆ 简化指令系统设计，适合超大规模集成电路实现；
- ◆ 提高机器的执行速度和效率；
- ◆ 降低设计成本，提高了系统的可靠性；
- ◆ 可以提供直接支持高级语言的能力，简化编译系统的设计。

➤ RISC技术不足

- ◆ 相对来说，RISC机器上的编译程序比CISC的机器编写难度大
- ◆ 指令少，某些复杂功能需要多条指令（相对CISC）：
 - ✓ 加重汇编负担，增加程序长度等；



2.5 操作数的类型和大小

➤ 指令系统关注的是：硬件直接认识的是哪些数据？能支持多大的数据以及多少种数据？

软件能处理的是什么？ -----软硬件的在数据层次上的分工问题

➤ 数据表示：计算机硬件能够直接识别、指令系统可以直接调用的数据类型。

- ◆ 所有数据类型中最常用、相对比较简单、用硬件实现比较容易的几种。

- ◆ 整型、浮点型、字符型等

➤ 数据结构：由软件进行处理和实现的各种数据类型。--链表，树、图等

- ◆ 研究：这些数据类型的逻辑结构与物理结构之间的关系，并给出相应的算法。 这需要软硬件共同支持。

- ◆ 数据结构要通过软件映像，变换成机器所具有的数据表示来实现

一般来说，除了数据表示之外的所有类型都是数据结构要研究的内容



2.5 操作数的类型和大小

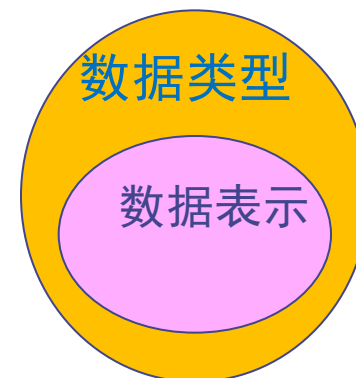


◆ 三者基本内容

- ✓ 数据类型：整数、字符、文件、图、表、阵列、栈、向量、树等
- ✓ 数据表示：定点数表示、逻辑数据表示、浮点数表示
- ✓ 数据结构：带有结构的数据元素的集合，串、队列、栈、树、图等
- ✓ **数据结构**：由软件进行处理和实现的各种数据类型。

◆ 三者关系

- ✓ 数据表示 $\subseteq$ 数据类型 数据结构 $\subseteq$ 数据类型
- ✓ 数据表示：硬件容易实现的简单数据类型
- ✓ 数据结构：软件实现，转换为数据表示





2.5 操作数的类型和大小

➤ 数据表示类型

- ✓ 当机器设置有定点算术运算一系列运算指令和相应的运算硬件，可以直接对定点数进行各种处理时，机器就有了定点数据表示（例如硬件有指令（比如 ADD）+ 硬件有电路（比如加法器）= 就有了某种数据表示。）。
- ✓ 同理整型、浮点型、字符型等
- ✓ 同理，如果不需要上述运算，硬件则可不设计

➤ 系统结构设计者要解决的问题：如何确定数据表示？（软硬件取舍折中的问题）

- ◆ 理论上只要软件能实现简单的简单的数据表示，例如定点数、字符等，那么复杂的数据类型可以通过软件来设计，例如浮点数的表示可以整数拼接。但大大降低了系统的性能效率等
- ◆ 反之复杂的数据类型融入在数据表示中，则硬件成本高，若很少用到，则设计不合理
- ◆ 如何进行软硬件取舍是确定数据表示的关键。



2.5 操作数的类型和大小

表示操作数类型的方法有**两种**

- ◆ 1、由指令中的操作码指定操作数的类型，例如对字节操作还是对字操作；例如add, R2, #3

定点数加	数据I	数据J
------	-----	-----

- ◆ 2、给数据加上标识，由数据本身给出操作数类型。

- ✓ **优点：**简化指令系统，可由硬件自动实现一致性检查和类型转换，缩小了机器语言与高级语言的语义差距，简化编译器等。
- ✓ **缺点：**由于需要在执行过程中动态检测标志符，动态开销比较大，所以采用这种方案的机器很少见
- ✓ 例如如下数据字符：标识符两位，可定义4种数据类型，三位可定义8种类型

标识符	数据
-----	----

数据字



2.5 操作数的类型和大小

➤ 操作数的大小：操作数的位数或字节数。

主要的大小：字节（8位）、半字（16位）

字（32位）、和双字（64位）

◆ 字符：用ASCII码表示，为一个字节大小。

◆ 整数：用二进制补码表示，其大小可以是字节、半字或单字。

◆ 浮点操作数：单精度浮点数（1个字）、双精度浮点数（双字）。

一般都采用IEEE 754浮点标准

◆ 十进制操作数类型

✓ 压缩十进制或二进制编码十进制（BCD码）：用4位二进制编码表示数字0~9，并将两个十进制数字合并到一个字节中存储。

✓ 非压缩十进制：将十进制数直接用字符串来表示。



2.6 MIPS指令系统结构

- 2.6.1 MIPS的寄存器
- 2.6.2 MIPS的数据表示
- 2.6.3 MIPS 的数据寻址方式
- 2.6.4 MIPS的指令格式
- 2.6.5 MIPS操作
- 2.6.6 MIPS控制指令
- 2.6.7 MIPS 的浮点操作



2.6 MIPS指令系统结构

MIPS64的一个子集，简称为MIPS。

- 1981年，斯坦福大学，研发了MIPS计算机，之后在此基础上形成了MIPS系列微处理器
- **MIPS计算机**：简单 64bit load-store 系统结构的 **RISC** 计算机
- **特征**：ALU指令的源和目标操作数都存放在寄存器中，只能通过Load和Store指令访问存储器，因此被称为Load-Store指令集结构；易于流水设计；寻址方式简单；操作简单



2.6.1 MIPS的寄存器

➤ **MIPS计算机**：简单 64bit load-store 系统结构的 RISC 计算机

MIPS 64 寄存器特点

➤ 32个64位通用寄存器（GPRs）

- ◆ R0, R1, ..., R31
- ◆ 也称为整数寄存器
- ◆ R0的值永远是0

➤ 一些特殊寄存器

- ◆ R0:恒为0; R31: 存放返回地址。
- ◆ 例如浮点状态寄存器：用来保存有关浮点操作结果的信息。

MIPS 64 寄存器特点

➤ 32个64位浮点数寄存器（FPRs）

- ◆ F0, F1, ..., F31
- ◆ 用来存放32个单精度浮点数（32位），也可以用来存放32个双精度浮点数（64位）。
- ◆ 存储单精度浮点数（32位）时，**只用到FPR的一半**，其另一半没用。



2. 6. 2 MIPS的数据表示

➤ MIPS的数据表示

◆ 整数

字节（8位） 半字（16位）

字（32位） 双字（64位）

◆ 浮点数

单精度浮点数（32位） 双精度浮点数（64位）

➤ MIPS的操作是针对64位整数以及32位或64位浮点数进行的。

➤ 字节、半字或者字在装入64位寄存器时 - load指令

◆ 用零扩展或者用符号位扩展来填充该寄存器的剩余部分。

◆ 装入以后，对它们将按照64位整数的方式进行运算。



2. 6. 3 MIPS的数据寻址方式

MIPS数据寻址方式的特点：

➤立即数寻址与偏移量寻址；立即数字段和偏移量字段都是16位的。

➤寄存器间接寻址是通过把0作为偏移量来实现的；

➤16位绝对寻址是通过把R0（其值永远为0）作为基址寄存器来完成的。

```
addi $r3, $r1, 100
```

注意： MIPS数据寻址方式

➤MIPS的存储器是按字节寻址的，地址为64位；

➤通过load和store指令来完成访存。

➤所有存储器访问都必须是边界对齐的。

➤寻址方式编码到操作码中。

➤操作码占6位。



2. 6. 4 MIPS的指令格式

MIPS数据寻址方式支持4种

- 寄存器寻址：如ADD R1, R2;
- 立即数寻址：ADD R1, R2, #42, #42表示立即数;
- 偏移寻址：ADD R1, R2, 42 (R3)
- 寄存器间接寻址：ADD R1, R2, 0 (R3), 0偏移量可以省略。

MIPS 3种指令格式

- 3种格式中，同名字段的位置固定不变。
- 固定长度，所有指令32位，操作码6位。
- 采用定长和扩展码相结合的方式，二者都是6位，例如R型指令。

I类指令；R类指令；J类指令



2.6.4 MIPS指令格式

➤ I类指令主要能完成功能

◆ 计算型指令

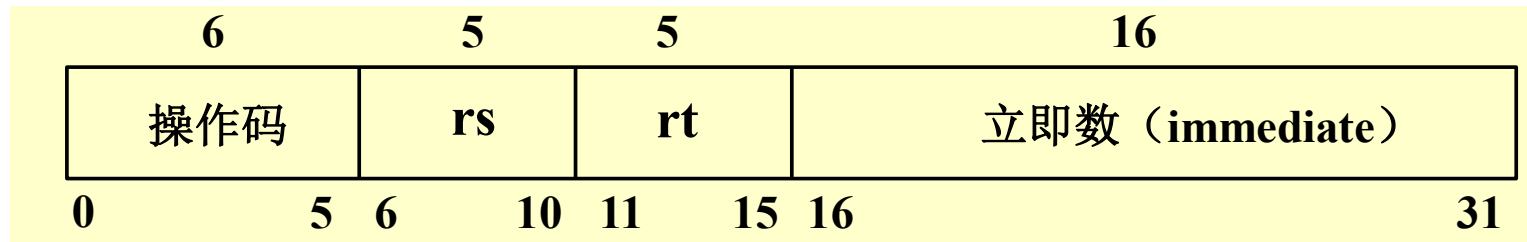
- ✓ 程序执行时，源操作数为 r_s 或者立即数，目标操作数为 r_t 。

◆ 存取数型指令

- ✓ 用于用于取数和存数操作，立即数：有符号数或无符号数，操作码根据不同指令而不同。

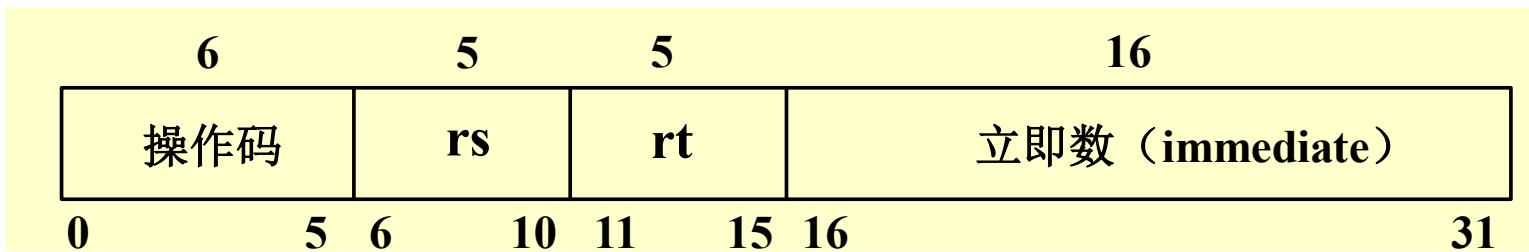
◆ 条件判断指令

- ✓ 涉及到条件分支的操作，使用1个寄存器和一个偏移量进行操作，根据 r_s 中的值进行条件分支操作。





- ◆ 包括所有的load和store指令，立即数指令，分支指令，寄存器跳转指令，寄存器链接跳转指令（程序外）。
 - ✓ 寄存器跳转指令（程序内，修改PC）
 - ✓ 寄存器链接跳转指令（程序外，例如函数调用）
- ◆ rs 源操作数寄存器编号 rt：目标寄存器变编号。
- ◆ 有一个操作数是立即数提供的。
- ◆ 立即数字段为16位用于提供立即数或偏移量（-32768---32767）。





2.6.4 MIPS指令格式

➤ I类指令

◆ load指令

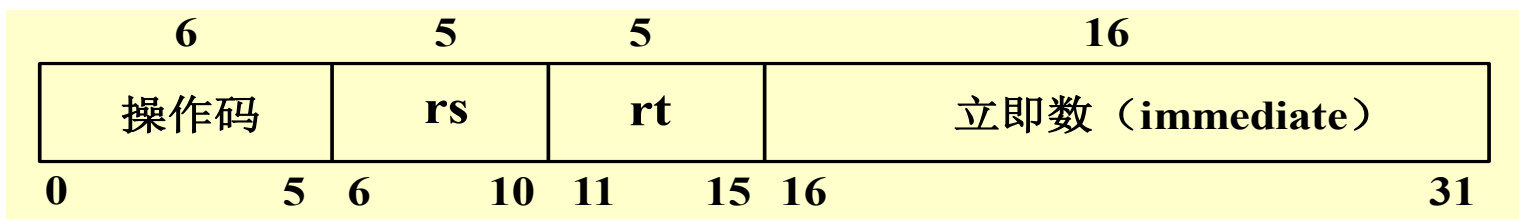
访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$

从存储器取来的数据放入寄存器 rt

◆ store指令

访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$

要存入存储器的数据放在寄存器 rt 中



load/store指令的寻址方式仅为寄存器偏移量寻址，需要指令提供立即数偏移量。

例如lw rt, 偏移量 (rs) 功能: $\text{rt} \leftarrow \text{memory}[\text{rs} + \text{偏移量}]$ rs为基址寄存器

注意: lw指令，偏移量应该符号扩展



2. 6. 4 MIPS指令格式

➤ I类指令

◆ 立即数指令

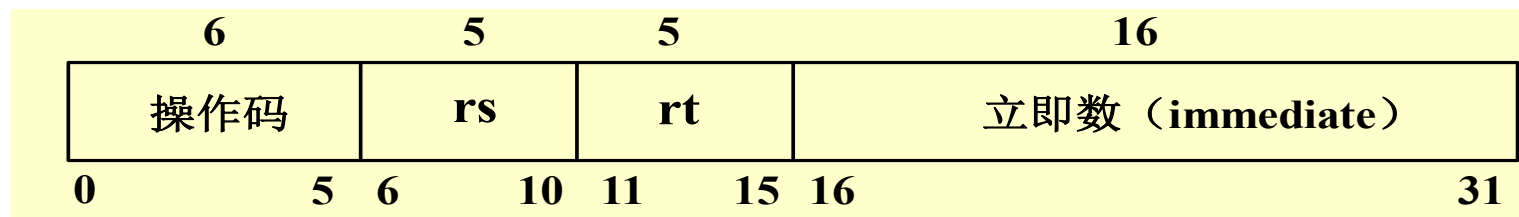
$\text{Regs}[\text{rt}] \leftarrow \text{Regs}[\text{rs}] \text{ op immediate}$ **ADDI \$r1, \$r2, 1**

◆ 分支指令/条件转移指令

转移目标地址：根据具体指令确定目标地址，表现形式不一

◆ 寄存器跳转、寄存器跳转并链接

转移目标地址为 $\text{Regs}[\text{rs}]$





2.6.4 MIPS指令格式

➤ I类指令---立即数指令

$\text{Regs}[\text{rt}] \leftarrow \text{Regs}[\text{rs}] \text{ op immediate}$

◆ 需要注意的是：MIPS所有寄存器都是64位，立即数是16位怎么办

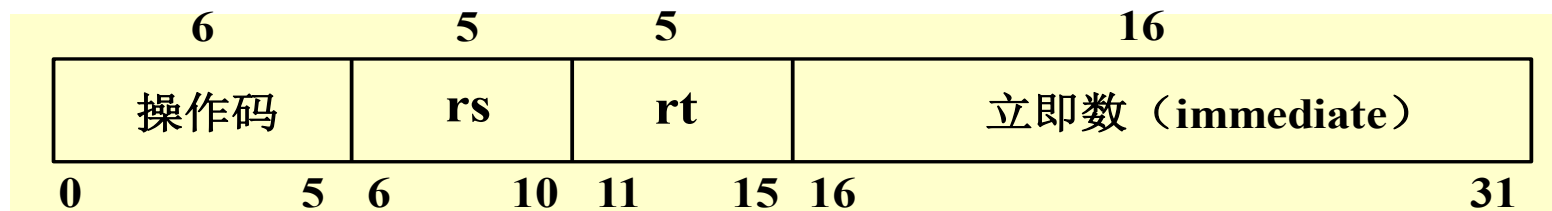
解决方法：扩展，怎么扩展

➤ 解决方法：扩展

◆ 运算之前，将立即数扩展为64位，扩展方法和运算有关

◆ 算术运算：符号扩展

◆ 逻辑运算，0扩展





2.6.4 MIPS指令格式

➤ I类访存指令

◆ load指令

访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$

从存储器取来的数据放入寄存器 rt

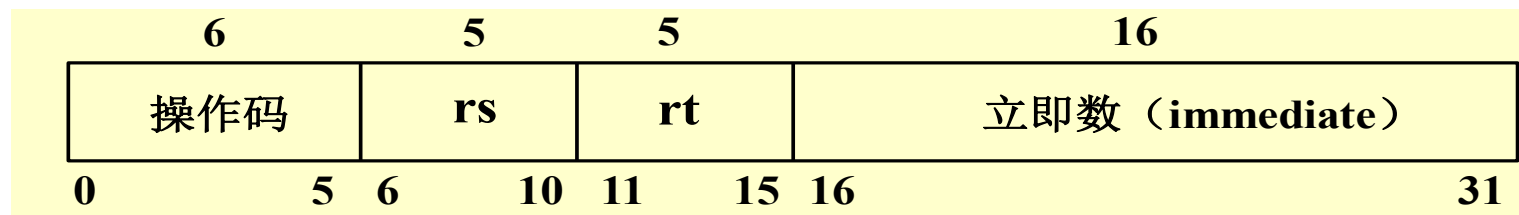
◆ store指令

访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$

要存入存储器的数据放在寄存器 rt 中

但是上述访存指令也涉及到立即数的加运算，怎么扩展

将立即数作为有符号数处理，也即偏移方向





I类指令---如果表示多于16位的数

立即数和偏移量的范围只有 **-32768 ~ +32767**。如果需要更大的数，编译器必须用多条指令（如 lui + ori）来合成。

6		5		5		16					
操作码		rs		rt		立即数 (immediate)					
0	5	6	10	11	15	16					31

andi rt, rs, imm 000 010 $rt \leftarrow rs \wedge imm$

ori rt, rs, imm 000 001 $rt \leftarrow rs \vee imm$

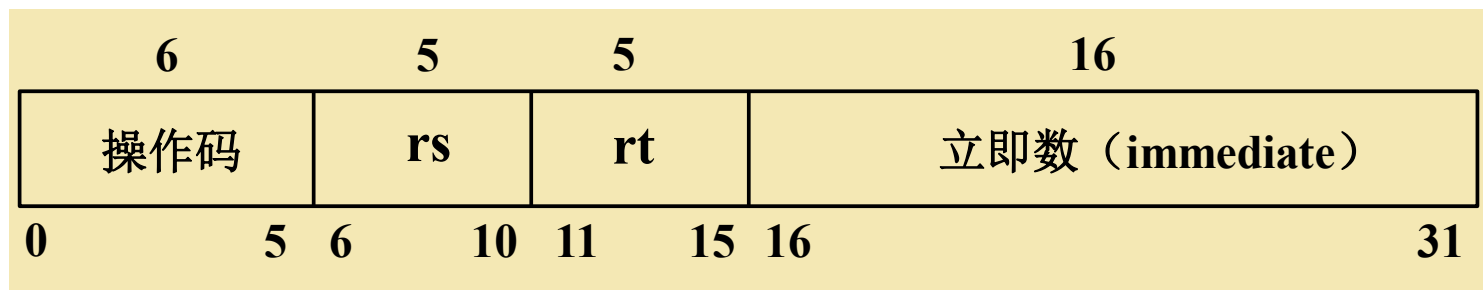
lui rt, imm 000 100 imm 需要怎么处理?

lui rt, imm: 把imm左移48位存入rt寄存器。与ori合作，可以为64位寄存器赋任意值：lui赋高32位，ori赋低32位

联系：完成同样功能，RISC指令拼接可完成CISC指令的功能。



I类指令---如何表示多于16位的数



lui rt, imm 000 100 该指令rs字段无用

lui rt, imm: 把imm左移48位存入rt寄存器。与ori合作，可以为64位寄存器赋任意值：lui赋高32位，ori赋低32位

例如：lui r1,1234H 执行后：r1=1234 0000 0000 0000H

ori r2,r1,5678H 执行后：r2=1234 0000 0000 5678H

注意 ori 指令执行过程：先将5678H进行0扩展

两条指令完成寄存器赋值操作



I类指令—分支指令的跳转

6	5	5	16
操作码	rs	rt	立即数 (immediate)
0	5	6	10
11	15	16	31

beq rt, rs, imm

过程: if (rs==rt)

PC←分支地址

else

PC←下一条指令地址 (PC+4)

分支地址计算 : 1、Sign Extension (符号扩展)

分支地址=PC+4+ SF⁴⁶##偏移量左移2位

2、##, 在硬件描述里代表左移2位 (乘以4)

联系程序: if (a==b)

o o o

else

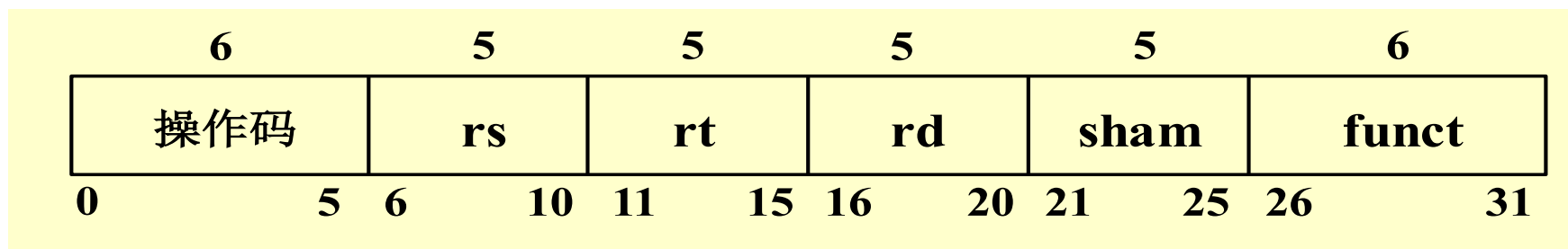
o o o



2.6.4 MIPS指令格式

➤ R类指令

- ◆ 包括ALU指令，专用寄存器读/写指令，move指令等。
- ◆ rs、rt分别表示源寄存器1和2，rd表示目标寄存器，
- ◆ sham字段用于移位操作，用于指定移位指令进行移位操作的位数，对于非移位指令，该域设为0
- ◆ ALU指令
 - ✓ $\text{Regs}[\text{rd}] \leftarrow \text{Regs}[\text{rs}] \text{ funct } \text{Regs}[\text{rt}]$
 - ✓ funct为具体的运算操作编码





◆只有寄存器型的操作数

◆操作码字段6位，皆为0，

func_t 字段与操作码结合，精确地指定指令的类型（思考二者为什么不合并？）。

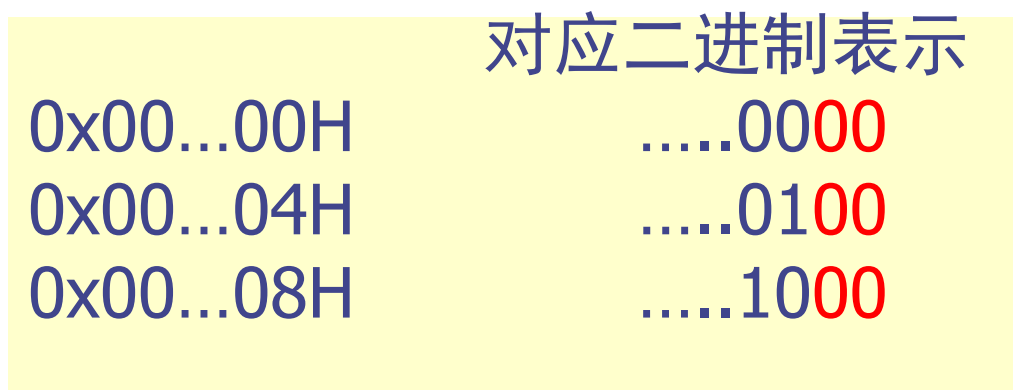
◆操作数与运算结果只能来自寄存器



注意R类指令操作码： 最高6位000000与funct结合确定指令功能



- ◆ 包括跳转指令，跳转并链接指令，自陷指令，异常返回指令。
- ◆ 在这类指令中，指令字的低26位是偏移量，左移两位，它与PC值相加形成跳转的地址。
 - ✓ 寻址空间0---64M 26左移2位—28位— $2^{28}=256\text{M}$ --- $256/4=64\text{M}$
 - ✓ 为什么左移两位（变为28位）？
(联系32位（4个字节）指令字长，整数边界存储，确保PC指向跳转指令起始地址)





J类指令



1、J next #next, 标号, 偏移量 功能: $PC \leftarrow \text{目标add}$

跳转目标地址 $\text{add} = PC + 4$ (高位) ## 偏移量



跳转目标地址 $\text{add} = PC + 4$ (高位) ## 偏移量左移两位



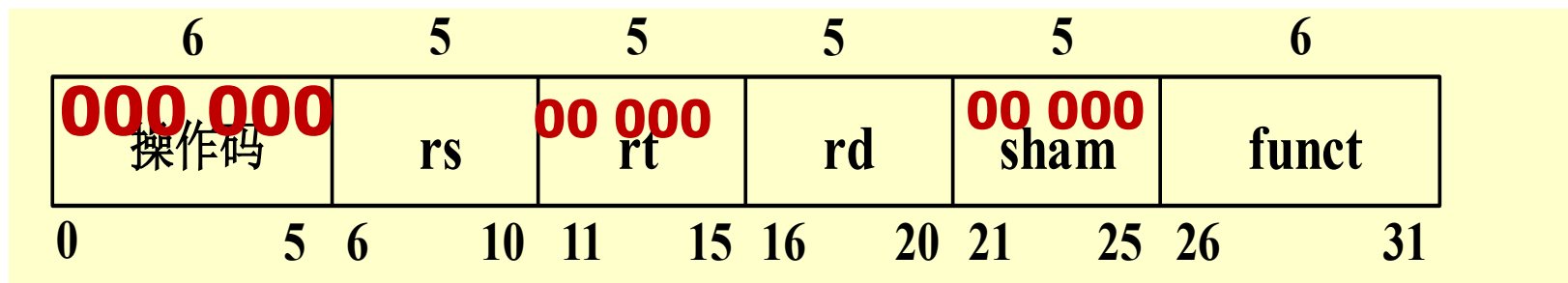
跳转目标地址 $\text{add} = PC + 4$ (高36位) ## 偏移量26位+左移两位



最终赋给 $PC = \text{目标add}$

>>> J类指令

Jr rs



***注意Jr rs 是R类指令，这里引入是为了和J类指令对比

2、Jr rs

#也是跳转指令，即将rs内容写入PC

如果指令指定rs是R31 例如Jr R31，则Jr和Jal 指令组合使用， 此时Jr是从子程序返回的指令

>>> J类指令



3、Jal next #next, 标号, 偏移量, 例如函数调用

指令功能: $R31 \leftarrow PC + 4$ $PC \leftarrow add$ (同上条指令)

过程同上, 最终跳转指令的目标pc=add 但



$R31 = PC + 4$ (当前跳转指令的下一条指令地址),

返回地址存放于R31

联系汇编中的函数调用, 子程序调用、中断、循环等

关于MIPS指令系统说法错误的是（ ）

- ☐ A MIPS属于精简指令系统
- ☒ B 三种类型的MIPS指令操作码长度各不相同
- ☐ C MIPS指令长度固定
- ☐ D MIPS指令的寻址方式蕴含在操作码中

提交



2. 6. 5 MIPS的操作

➤ MIPS指令操作可以分为四大类

- ◆ load和store

- ◆ ALU操作

- ◆ 分支与跳转

- ◆ 浮点操作

➤ 符号的意义

- ◆ $x \leftarrow_n y$: 从y传送n位到x

- ◆ $x, y \leftarrow z$: 把z传送到x和y

下面选项中，MIPS指令不包含哪类指令？

- ☐ A Load和store
- ☐ B ALU操作
- ☐ C 分支与跳转
- ☒ D 向量指令

提交



2.6.5 MIPS指令操作

➤ MIPS指令操作可以分为四大类

- ◆ load和store: 可以对MIPS的所有寄存器和浮点寄存器进行load（载入）和store（存储）操作，但对通用寄存器R0的load操作没有任何效果

➤ 符号的意义

- ◆ $x \leftarrow_n y$: 从y传送n位到x
- ◆ $x, y \leftarrow z$: 把z传送到x和y

- ◆ 下标: 表示字段中具体的位;

- ✓ 对于指令和数据，按从最高位到最低位（即从左到右）的顺序依次进行编号，最高位为第0位，次高位为第1位，依此类推。
- ✓ 下标可以是一个数字，也可以是一个范围。

例如: $\text{Regs}[\text{R4}]_0$: 寄存器R4中内容的符号位

$\text{Regs}[\text{R4}]_{56-63}$: 共8位，寄存器R4内容的最低一个字节

- ◆ Mem: 表示主存;

- ✓ 按字节寻址，可以传输任意个字节。

- ◆ 上标: 用于表示对字段进行复制的次数。

例如: 0^{32} : 一个32位长的全0字段



2. 6. 5 MIPS指令操作

➤ MIPS指令符号的意义：

- ◆ **符号##**：用于两个字段的拼接，并且可以出现在数据传送的任何一边。

举例：R8、R10：64位的寄存器，则

$$\text{Regs}[\text{R8}]_{32-63} \leftarrow_{32} (\text{Mem}[\text{Regs}[\text{R6}]]_0)^{24} \text{##} \text{Mem}[\text{Regs}[\text{R6}]]$$

表示的意义：以R6的内容作为地址访问内存，得到的**字节**按符号位扩展为32位后存入R8的低32位，R8的高32位（即 $\text{Regs}[\text{R8}]_{0-31}$ ）不变。



2. 6. 5 MIPS指令操作

$\text{Regs}[\text{R8}]_{32-63} \xleftarrow{32} (\text{Mem}[\text{Regs}[\text{R6}]]_0)^{24} \quad \#\# \text{Mem}[\text{Regs}[\text{R6}]]$

表示意义：箭头表示数据传输，

$\text{Regs}[\text{R8}]_{32-63}$ 表示：R8寄存器的低32位内容

$(\text{Mem}[\text{Regs}[\text{R6}]]_0)^{24}$ 表示R6的值作为访存地址所得到的内存单元内容的最高位（符号位）并将其复制24次

$\#\# \text{Mem}[\text{Regs}[\text{R6}]]$ ：表示以R6寄存器间接寻址的存储单元对应内容的低8位



2. 6. 5 MIPS指令操作

MIPS加载和存储指令的命名规则如下：

- 1) 首字母L表示加载 (Load) , S表示存储 (Store)
- 2) 首字母U表示无符号 (Unsigned) , 其余默认为有符号 (Signed)
- 3) 尾字母R表示右 (Right) , L表示左 (Left)
- 4) 字母B示字节 (Byte) , H表示半字 (Halfword) , W表示字 (Word)



MIPS指令举例--load和store指令

指令举例	指令名称	含义
LD R2, 20 (R3)	装入双字	$\text{Regs}[\text{R2}] \leftarrow_{64} \text{Mem}[\text{20} + \text{Regs}[\text{R3}]]$
LW R2, 40 (R3)	装入字	$\text{Regs}[\text{R2}] \leftarrow_{64} (\text{Mem}[\text{40} + \text{Regs}[\text{R3}]]_0)^{32} \text{##}$ Mem[40+Regs[R3]] (从LW可知取32位)
LB R2, 30 (R3)	装入字节	$\text{Regs}[\text{R2}] \leftarrow_{64} (\text{Mem}[\text{30} + \text{Regs}[\text{R3}]]_0)^{56} \text{##}$ Mem[30+Regs[R3]]
LBU R2, 40 (R3)	装入无符号字节	$\text{Regs}[\text{R2}] \leftarrow_{64} 0^{56} \text{## Mem}[\text{40} + \text{Regs}[\text{R3}]]$
LH R2, 30 (R3)	装入半字	$\text{Regs}[\text{R2}] \leftarrow_{64} (\text{Mem}[\text{30} + \text{Regs}[\text{R3}]]_0)^{48} \text{##}$ Mem[30+Regs[R3]]## Mem[31+Regs[R3]] 连续读取两个字节
L.S F2, 60 (R4)	装入单精度浮点数	$\text{Regs}[\text{F2}] \leftarrow_{64} \text{Mem}[\text{60} + \text{Regs}[\text{R4}]] \text{## } 0^{32}$
L.D F2, 40 (R3)	装入双精度浮点数	$\text{Regs}[\text{F2}] \leftarrow_{64} \text{Mem}[\text{40} + \text{Regs}[\text{R3}]]$
SD R4, 300 (R5)	保存双字	$\text{Mem}[\text{300} + \text{Regs}[\text{R5}]] \leftarrow_{64} \text{Regs}[\text{R4}]$
SW R4, 300 (R5)	保存字	$\text{Mem}[\text{300} + \text{Regs}[\text{R5}]] \leftarrow_{32} \text{Regs}[\text{R4}]$
S.S F2, 40 (R2)	保存单精度浮点数	$\text{Mem}[\text{40} + \text{Regs}[\text{R2}]] \leftarrow_{32} \text{Regs}[\text{F2}]_{0..31}$
SH R5, 502 (R4)	保存半字	$\text{Mem}[\text{502} + \text{Regs}[\text{R4}]] \leftarrow_{16} \text{Regs}[\text{R5}]_{48..63}$



3. load和store指令

指令举例	指令名称	含 义
LD R2, 20 (R3)	装入双字	$\text{Regs}[\text{R2}] \leftarrow_{64} \text{Mem}[\text{20} + \text{Regs}[\text{R3}]]$
LW R2, 40 (R3)	装入字	$\text{Regs}[\text{R2}] \leftarrow_{64} (\text{Mem}[\text{40} + \text{Regs}[\text{R3}]]_0)^{32} \text{##}$ $\text{Mem}[\text{40} + \text{Regs}[\text{R3}]]$ 取Mem[40+Regs[R3]] 内容的32位，符号位扩展32位，共64位
LB R2, 30 (R3)	装入字节	$\text{Regs}[\text{R2}] \leftarrow_{64} (\text{Mem}[\text{30} + \text{Regs}[\text{R3}]]_0)^{56} \text{##}$ $\text{Mem}[\text{30} + \text{Regs}[\text{R3}]]$
		取出Mem[30+Regs[R3]] 对应的低8位内容，然后做符号位扩展 (Mem[30+Regs[R3]] ₀) ⁵⁶ ，56位符号位扩展
SH R5, 502 (R4)	保存半字	$\text{Mem}[\text{502} + \text{Regs}[\text{R4}]] \leftarrow_{16} \text{Regs}[\text{R5}]_{48..63}$

3. load和store指令

指令举例	指令名称	含义
LBU R2, 40(R3)	装入无符号字节	$\text{Regs}[\text{R2}] \leftarrow_{64} 0^{56} \text{ \#\# Mem}[40+\text{Regs}[\text{R3}]]$ 0^{56} 复制56个0即0扩展，取 $\text{Mem}[40+\text{Regs}[\text{R3}]]$ 的低8位
LH R2, 30(R3)	装入半字	$\text{Regs}[\text{R2}] \leftarrow_{64} (\text{Mem}[30+\text{Regs}[\text{R3}]]_0)^{48} \text{ \#\# Mem}[30+\text{Regs}[\text{R3}]] \text{ \#\# Mem}[31+\text{Regs}[\text{R3}]]$ 装入半字：需要访问两个字节，30, 31为相对寄存器内容的偏移量，因为内存按字节编址，最后以 $\text{Mem}[30+\text{Regs}[\text{R3}]]_0$ 这个字节对应的符号位扩展48位
L.S R2, 60(R4)	装入单精度浮点数	$\text{Regs}[\text{R2}] \leftarrow_{64} \text{Mem}[60+\text{Regs}[\text{R4}]] \text{ \#\# } 0^{32}$ 从存储器中读取单精度浮点数到寄存器中
L.D F2, 40(R3)	装入双精度浮点数	$\text{Regs}[\text{F2}] \leftarrow_{64} \text{Mem}[40+\text{Regs}[\text{R3}]]$
SD R4, 300(R5)	保存双字	$\text{Mem}[300+\text{Regs}[\text{R5}]] \leftarrow_{64} \text{Regs}[\text{R4}]$
SW R4, 300(R5)	保存字	$\text{Mem}[300+\text{Regs}[\text{R5}]] \leftarrow_{32} \text{Regs}[\text{R4}]$
S.S F2, 40(R2)	保存单精度浮点数	$\text{Mem}[40+\text{Regs}[\text{R2}]] \leftarrow_{32} \text{Regs}[\text{F2}]_{0..31}$
SH R5, 502(R4)	保存半字	$\text{Mem}[502+\text{Regs}[\text{R4}]] \leftarrow_{16} \text{Regs}[\text{R5}]_{48..63}$

4. ALU指令

寄存器—寄存器型（RR型）指令或立即数型
算术和逻辑操作：加、减、与、或、异或和移位等

指令举例	指令名称	含义
DADDU R1, R2, R3	无符号加	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R2}] + \text{Regs}[\text{R3}]$
DADDIU R4, R5, #6	加无符号立即数	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R5}] + 6$
LUI R1, #4	把立即数装入到一个字的高16位	$\text{Regs}[\text{R1}] \leftarrow 0^{32} \text{ ## } 4 \text{ ## } 0^{16}$ 把一个16位的立即数装入到寄存器的高16位，低16位补0
DSLL R1, R2, #5	双字逻辑左移，同理右移	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R2}] \ll 5$
DSLT R1, R2, R3	置小于	If ($\text{Regs}[\text{R2}] < \text{Regs}[\text{R3}]$) $\text{Regs}[\text{R1}] \leftarrow 1$ else $\text{Regs}[\text{R1}] \leftarrow 0$



2. 6. 5 MIPS指令操作

R0的值永远是0，它可以用来合成一些常用的操作。

例如：

```
DADDIU R1, R0, #100
```

给寄存器R1装入常数100

```
DADD R1, R0, R2
```

把寄存器R2中的数据传送到寄存器R1



2.6.6 MIPS的控制指令

➤ 跳转指令

- ◆ 根据跳转指令确定目标地址的方式不同以及跳转时是否链接，可以把跳转指令分成4种。

➤ 由一组跳转和一组分支指令来实现控制流的改变

◆ 确定目标地址的方式

- ✓ 把指令中的26位偏移量左移2位（因为指令字长都是4个字节，**联系哪一部分知识？**）后，替换程序计数器的低28位；
- ✓ **间接跳转**：由指令中指定的一个寄存器来给出转移目标地址。

➤ 典型的MIPS控制指令

◆ 跳转的两种类型

- ✓ **简单跳转**：把目标地址送入程序计数器。
- ✓ **跳转并链接（带AL的指令）**：把目标地址送入程序计数器，把返回地址（即顺序下一条指令的地址）放入寄存器R31。

跳转指令

指令举例	指令名称	含义
J name	跳转 ， 无条件跳转指令	$PC_{36..63} \leftarrow name$ 例如 J 0X1000 #跳转到内存地址0x1000
JAL name	跳转并链接 带链接的无条件跳转指令， 相当于调用	$Regs[R31] \leftarrow PC+4$; $PC_{36..63} \leftarrow name$; $((PC+4) - 2^{27}) \leq name < ((PC+4) + 2^{27})$ 1、先将下一条指令保存R31； 2、然后跳转到目标地址；
JALR R3	寄存器跳转并链接	$Regs[R31] \leftarrow PC+4$; $PC \leftarrow Regs[R3]$
JR R5	寄存器跳转	$PC \leftarrow Regs[R5]$
BEQZ R4, name	等于零时分支	if (Regs[R4]== 0) $PC \leftarrow name$; $((PC+4) - 2^{17}) \leq name < ((PC+4) + 2^{17})$
BNE R3, R4, name	不相等时分支	if (Regs[R3] != Regs[R4]) $PC \leftarrow name$ $((PC+4) - 2^{17}) \leq name < ((PC+4) + 2^{17})$
MOVZ R1, R2, R3	等于零时移动	if (Regs[R3]==0) $Regs[R1] \leftarrow Regs[R2]$

跳转指令

指令举例	指令名称	含义
J name	跳转 ， 无条件跳转指令	$PC_{36..63} \leftarrow name$ 1、Name 跳转的目标内存地址，26位立即数，无符号整数，表示跳转地址的高4位和低28位，共一个字节的地址。 2、其中MIPS指令地址以字节为单位，因此26位跳转地址必须是4的倍数（立即数左移2位，保证起始地址是**** ...*00）。 例如 J 0X1000 #跳转到内存地址0x1000
JAL name	跳转并链接 带链接的无条件跳转指令	$Regs[R31] \leftarrow PC+4$ ； $PC_{36..63} \leftarrow name$ ； $((PC+4) - 2^{27}) \leq name < ((PC+4) + 2^{27})$ 1、先将下一条指令保存到R31； 2、然后跳转到目标地址；
JALR R3	寄存器跳转并链接	$Regs[R31] \leftarrow PC+4$ ； $PC \leftarrow Regs[R3]$
JR R5	寄存器跳转	$PC \leftarrow Regs[R5]$
BEQZ R4, name	等于零时分支	if (Regs[R4]== 0) $PC \leftarrow name$ ； $((PC+4) - 2^{17}) \leq name < ((PC+4) + 2^{17})$
BNE R3, R4, name	不相等时分支	if (Regs[R3] != Regs[R4]) $PC \leftarrow name$ $((PC+4) - 2^{17}) \leq name < ((PC+4) + 2^{17})$
MOVZ R1, R2, R3	等于零时移动	if (Regs[R3]==0) $Regs[R1] \leftarrow Regs[R2]$

跳转指令

指令举例	指令名称	含义
J name	跳转	$PC_{36..63} \leftarrow name$ 例如 J 0X1000 #跳转到内存地址0x1000
JAL name	跳转并链接	$Regs[R31] \leftarrow PC+4$; $PC_{36..63} \leftarrow name$; $((PC+4) - 2^{27}) \leq name < ((PC+4) + 2^{27})$
JALR R3	寄存器跳转并链接	$Regs[R31] \leftarrow PC+4$; $PC \leftarrow Regs[R3]$
JR R5	寄存器跳转	$PC \leftarrow Regs[R5]$
BEQZ R4, name	等于零时分支	if ($Regs[R4] == 0$) $PC \leftarrow name$; $((PC+4) - 2^{17}) \leq name < ((PC+4) + 2^{17})$
BNE R3, R4, name	不相等时分支	if ($Regs[R3] \neq Regs[R4]$) $PC \leftarrow name$ $((PC+4) - 2^{17}) \leq name < ((PC+4) + 2^{17})$ 标签name的地址范围由当前PC加上4字节对齐后的地址决定
MOVZ R1, R2, R3	等于零时移动	if ($Regs[R3] == 0$) $Regs[R1] \leftarrow Regs[R2]$



2.6.7 MIPS的浮点操作

➤ 分支指令（条件转移）

- ◆ 分支条件由指令确定

例如：测试某个寄存器的值是否为零

- ◆ 提供一组比较指令，用于比较两个寄存器的值。

例如：“置小于”指令

- ◆ 有的分支指令可以直接判断寄存器内容是否为负，或者比较两个寄存器是否相等。

- ◆ 分支的目标地址

由16位带符号偏移量左移两位后和PC相加的结果来决定

- ◆ 一条浮点条件分支指令：通过测试浮点状态寄存器来决定是否进行分支。



2.6.7 MIPS的浮点操作

➤ 由操作码指出操作数是单精度（SP）或双精度（DP）

◆ 后缀S：表示操作数是单精度浮点数

◆ 后缀D：表示是双精度浮点数

➤ 浮点操作

包括加、减、乘、除，分别有单精度和双精度指令。

➤ 浮点数比较指令

◆ 根据比较结果设置浮点状态寄存器中的某一位，以便于后面的分支指令BC1T（若真则分支）或BC1F（若假则分支）测试该位，以决定是否进行分支。